

Bibliotecas. Pruebas. Recursividad

Juan Arnedo Moreno

Programación básica (ASX)
Programación (DAM)
Programación (DAW)

Índice

Introducción	5
Resultados de aprendizaje	7
1 Descomposición en clases y bibliotecas	9
1.1 Programas con múltiples clases	9
1.1.1 ¿Qué es realmente una clase?	10
1.1.2 Estructura de un programa modular en Java	11
1.1.3 Definición y uso de clases adicionales	12
1.1.4 Ventajas de la modularidad usando clases adicionales	16
1.2 Bibliotecas de clases: packages	17
1.2.1 Definición de packages	18
1.2.2 Criterios para crear bibliotecas de clases	19
1.2.3 Uso de clases de otros packages	22
1.2.4 Estructura de los archivos dentro de los packages	24
1.3 La API del lenguaje Java	25
1.3.1 Uso de clases dentro de las bibliotecas de Java	26
1.3.2 Inicialización con parámetros	28
1.4 Métodos estáticos	30
1.4.1 La clase Math	31
1.4.2 La clase Arrays	33
1.5 Documentación de programas en Java	36
1.5.1 Javadoc	37
1.5.2 Sintaxis general	38
1.5.3 Palabras clave	39
1.6 Solución de los retos propuestos	41
2 Creación de una aplicación modular. El juego de combates en la arena	45
2.1 El juego de combates en la arena	45
2.1.1 Descripción detallada del programa	46
2.1.2 Identificación de los datos a tratar	49
2.1.3 Diseño descendente	49
2.1.4 Módulos	51
2.2 La biblioteca "juego.arena.reglas"	52
2.2.1 La clase Monedas	52
2.2.2 La clase Luchador	53
2.2.3 La clase Bestiario	59
2.2.4 La clase Combate	60
2.3 La biblioteca "juego.arena.interfaz"	63
2.3.1 La clase EntradaTeclado	63
2.3.2 La clase SalidaPantalla	64
2.4 La clase principal	65
2.5 Simplificación de algoritmos complejos usando recursividad	67

2.5.1 Aplicación de la recursividad	68
2.5.2 Implementación de la recursividad	69
2.5.3 Recursividad al juego de combates en la arena	72

Introducción

Ahora mismo, se puede considerar que el desarrollo de programas bien estructurados se fundamenta en una serie de recursos básicos:

- La organización de los valores a tratar mediante tipos de datos apropiados.
- El uso de las estructuras secuencial, condicional y repetitiva.
- La descomposición de problemas mediante diseño descendente.

Uno de los recursos más importantes es el último, la descomposición del problema, puesto que si se realiza correctamente permite reutilizar un bloque de código, en forma de método, en diferentes partes del programa sin tener que escribirlo repetidas veces. En este sentido, una herramienta muy interesante sería poder aplicar esta reutilización, no sólo en un mismo programa, sino en cualquier programa. Una vez resuelto un subproblema para un caso concreto, poder usar los métodos resultantes en otros programas diferentes sin tener que hacer "copiar y pegar" del código. A medida que vaya resolviendo programas, le resulta más fácil hacer el código de nuevos.

Otro aspecto a tener en cuenta vinculado a la descomposición del problema es el hecho de que, en los programas complejos, el número de métodos resultantes puede ser bastante grande, lo que puede hacer que el archivo de su programa crezca bastante hasta el punto de ser difícil de leer o seguir. Quizás también sería interesante disponer de mecanismos para ordenar los métodos según algún criterio (temática, tipos de parámetros, etc.), de la misma manera que es más pulido poder ordenar una serie de fotos en carpetas etiquetadas, en lugar de tenerlas todas en una sola.

En el apartado Programación modular se presenta el mecanismo que proporcionan los lenguajes estructurados para poder disponer de esta posibilidad, mediante la realización de un diseño modular de las aplicaciones. Así pues, aprenderá los conceptos de módulo aplicables a la mayoría de lenguajes, y los llevará a la práctica en el lenguaje Java. Las grandes aplicaciones se trocean en módulos, cada uno de los cuales tiene una funcionalidad determinada. Así, en una aplicación para una gestión integral de un centro docente, podríamos tener el módulo que gestione al personal que trabaja en el centro, el módulo que gestiona el alumnado, el módulo que gestiona los horarios, etc. La lista podría irse ampliando. Estos módulos tienen cierta autonomía, pero necesitan mantener relaciones con el resto de módulos que forman la aplicación.

Una vez que dispone ya de todas las herramientas para generar programas complejos, en el apartado "Bibliotecas. Pruebas. Recursividad" se muestran una serie de técnicas complementarias para poder garantizar que estos son robustos y bien documentados. También se introducirá brevemente el concepto de recursividad, un mecanismo avanzado para poder simplificar la codificación de ciertos tipos de problemas complejos.

Para conseguir nuestros objetivos, es especialmente importante reproducir en el su entorno de trabajo todos los ejemplos incorporados en el texto, las actividades y los ejercicios de autoevaluación.

Resultados de aprendizaje

Al finalizar esta unidad, el alumno/a:

1. Prueba, depura, comenta y documenta los programas.
2. Define el concepto de librerías y su utilidad.
3. Utiliza librerías en la elaboración de programas
4. Conoce las nociones básicas de la recursividad y sus aplicaciones clásicas.

1. Descomposición en clases y bibliotecas

A la hora de crear una unidad didáctica o un libro, estructurarla en apartados o capítulos permite que sea mucho más fácil plantear sus contenidos, desde el punto de vista del autor, y seguir o comprender, desde el punto de vista del lector (o el propio autor, si en el futuro quiere realizar enmiendas). Ahora bien, por muy ordenada que sea esta estructura, a medida que crece el tamaño del documento, tarde o temprano llegará el momento en que habrá que plantearse si más bien lo necesario es empezar una nueva unidad o un nuevo volumen. Un documento demasiado largo se hace pesado y difícil de gestionar. Cuando esto sucede, normalmente la división se planteará de forma que cada unidad o volumen corresponda a una temática y resultados de aprendizaje

concretos.

En este sentido, y como ya ha visto, un programa no es muy distinto. Su subdivisión en diferentes métodos a partir de la aplicación del diseño descendente permite tener código aseado y pulido, mucho más fácil tanto de crear como de entender.

Adicionalmente, también permite reusar código útil en diferentes partes del programa, en lugar de tener que reescribirlo varias veces. Ahora bien, también del mismo modo, si el programa es complejo y acaba requiriendo de muchos métodos, la cantidad de métodos de todo tipo y el tamaño de su archivo también puede llegar a ser considerable.

Una solución que permite muchos lenguajes de programación es seguir exactamente el camino de la analogía planteada: dividir el código del programa en diferentes partes, de acuerdo a una temática concreta.

Un programa modular es aquél que se ha desarrollado dividiéndolo en componentes fáciles de modificar e intercambiar, llamados módulos. Cada módulo agrupa un conjunto de funciones o métodos que realizan tareas relacionadas.



Un programa monolítico es como un bloque de piedra, hecho de una pieza.
Fuente: Richard-sr

En contraposición a un programa modular, se encuentran los programas monolíticos, en los que todo el código se concentra en un único módulo. Normalmente, en el caso de programas complejos, se suele considerar un mal hábito realizar programas monolíticos.

1.1 Programas con múltiples clases

La forma más directa de hacer un programa modular es establecer una correspondencia uno a uno entre módulos y archivos donde está escrito el código fuente de un programa. Por tanto, cada módulo se representa con un archivo de código fuente diferente. Esta aproximación puede utilizarse en muchos lenguajes de programación. En el caso de Java, un programa modular está compuesto de la agregación de múltiples clases, en lugar de sólo una como ha trabajado hasta ahora.

1.1.1 ¿Qué es realmente una clase?

Antes de ver cómo es posible realizar un programa modular, vale la pena recapitular sobre el concepto de “clase” y su aplicación dentro de los programas que se desarrollan en Java. Si bien una definición estricta y formal de este concepto está muy vinculada a la orientación a objetos, y por tanto más allá del ámbito de este temario, si se recoge de todos los golpes que se ha usado desde una perspectiva práctica, se dará cuenta de que ha sido para referirse a tres cosas.

- Un programa en Java. Por un lado, los archivos de sus programas, propiamente, son clases (al fin y al cabo, se inician con la declaración `public class...`). Al aplicar diseño descendente, su código queda distribuido en un método principal (`main`), que indica su primera instrucción y el punto de inicio de su flujo de control, junto a distintos métodos adicionales que pueden ser invocados directamente.
- Un repositorio de métodos. Por otra parte, también se ha utilizado el término clase para referirse a bibliotecas de métodos, que actúan como extensiones en las instrucciones disponibles por defecto en el lenguaje. Antes de poder hacerlo, sin embargo, es necesario inicializarlas correctamente. El ejemplo más claro es la clase `Scanner`, que ofrece un repertorio de métodos para controlar la lectura de datos desde el teclado (`nextLine()`, `nextInt()`, `parseFloat()`, etc.).
- Un tipo compuesto. Por último, este mismo término se ha usado como sinónimo de tipo compuesto. En este caso, el ejemplo por antonomasia es la clase `String`, utilizada para referirse a cadenas de texto dentro de Java. Los tipos compuestos de Java permiten manipular datos complejos mediante la invocación de métodos (`charAt(...)`, `indexOf(...)`, etc.).

El código fuente de `Scanner` y `String` está disponible en la página de [descargas](#) de Oracle (Java SE 6 JDK Source Code). Ahora bien, ¿no es sencillo de entender!

Si se fija, en los tres casos hay un factor común. Todos ellos disponen de un conjunto de métodos que es posible invocar. Y es que, en realidad, a pesar de sus diferencias en el contexto bajo el que se usan y la forma de invocar sus métodos, los tres casos son exactamente lo mismo en última instancia: código fuente dentro de un archivo llamado `NomClase.java`, con la declaración `public class NomClase...`, y estructurado como una serie de métodos declarados en su interior.

Por tanto, aunque no sepa exactamente cuál es su código fuente al completo, está claro que la clase `Scanner` ha sido desarrollada por alguien, dentro de un archivo llamado `Scanner.java`, y una parte de su código es:

```
1 public class Scanner { //Otras declaraciones
2     (constantes, vars. globales...) ... public String nextLine() { //Código del método ...
3
4
5
6     } public int nextInt() {
7         //Código del método ...
8     }
9     //Otros métodos ...
10 }
```

Lo mismo ocurre con la clase String (con las definiciones de sus propios métodos):

```
1 public class String { //Otras declaraciones
2     (constantes, vars. globales...) ... public int indexOf(String textACercar){
3
4         //Código del método ...
5
6     } public int length() {
7         //Código del método ...
8     }
9     //Otros métodos ...
10 }
```

1.1.2 Estructura de un programa modular en Java

Una vez que se ha visto que todas las clases, independientemente de su rol, en realidad se codifican de la misma manera (como archivos que contienen una serie de métodos que pueden ser invocados), es el momento de ver cómo se relacionan a escala general para generar un programa modular.

Ante todo, se recapitulará también sobre qué es un programa, previendo todos los nuevos elementos que ha ido aprendiendo. Recuerde que un programa no es más que una secuencia ordenada de instrucciones que se van ejecutando de inicio a fin. Esta secuencia tiene ciertas particularidades, puesto que mediante las estructuras de control (selección y repetición), es posible crear bifurcaciones o bucles en la secuencia. Además, para programas complejos, también se pueden encontrar invocaciones a métodos, haciendo posible distribuir las instrucciones en distintos bloques, que pueden ser ejecutados repetidas veces en diferentes momentos del proceso. Todos estos métodos están escritos en un mismo archivo, que contiene el código fuente de todo el programa. De entre todos los métodos, se encuentra el principal (main), el cual indica cuál es la primera instrucción.

Pues bien, la única diferencia de un programa modular en Java es que los métodos, en lugar de estar escritos todos en el mismo archivo, están distribuidos dentro de diferentes archivos (diferentes clases), tal y como se contrasta en la figura 1.1. Fíjese que sólo cambia dónde están escritos físicamente los métodos, pero el flujo de control, con el orden de ejecución de las instrucciones e invocación de los métodos, es exactamente el mismo.

En este ejemplo, el programa modular se compondría de tres archivos diferentes, llamados Principal.java, Modul1.java y Modul2.java.

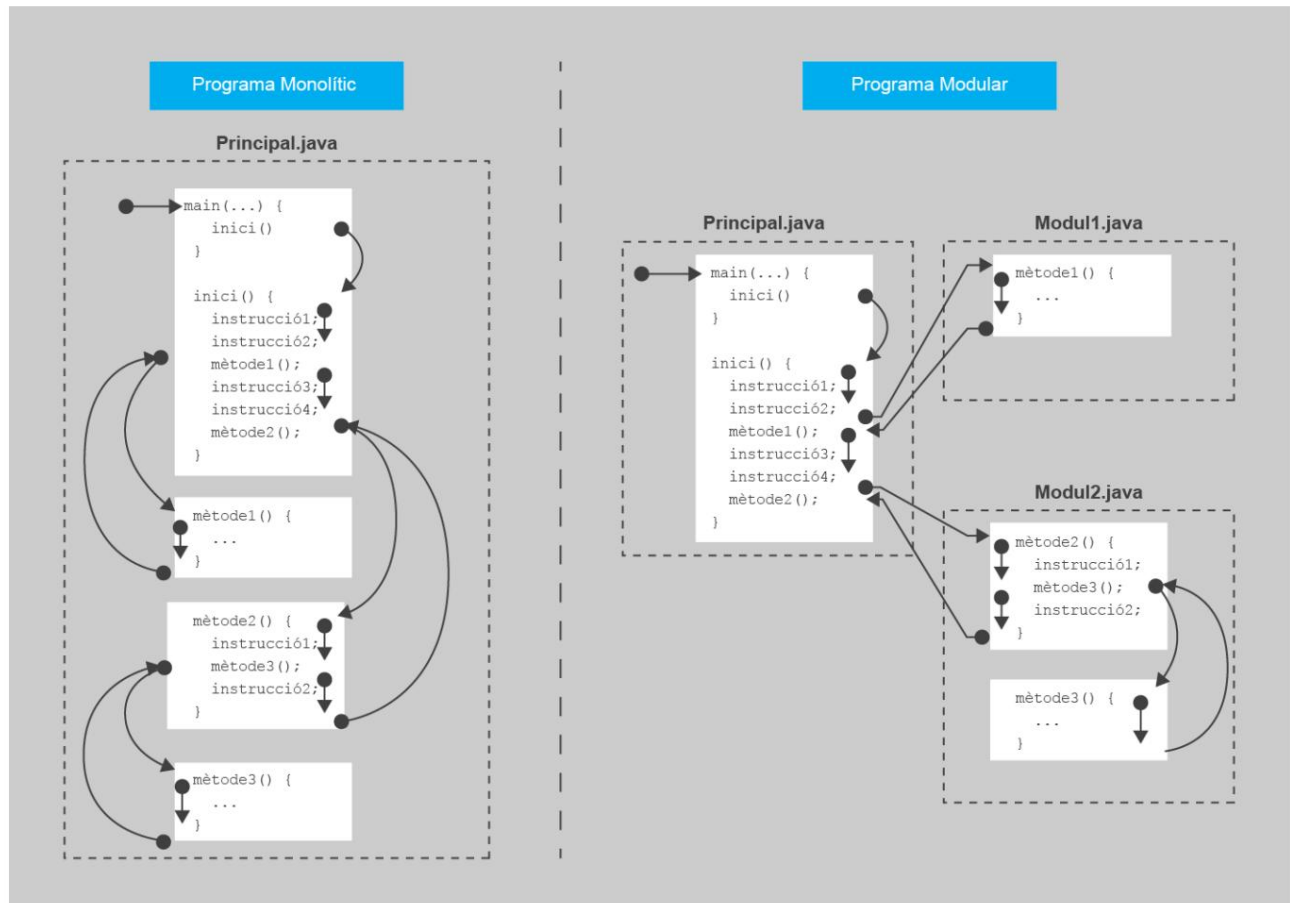
Normalmente, la escritura de métodos en ficheros separados no es arbitraria, sino que suele realizarse de acuerdo a algún criterio de orden, como agrupar métodos por funcionalidades similares (una clase con los métodos relacionados con operar con arrays, otra con los que leen datos desde el teclado, etc.). Por tanto, el quid de la cuestión para hacer un programa modular es ser capaz de decidir con criterio cómo distribuir los diferentes métodos en diferentes clases (y en cuántas) y saber cómo invocar métodos declarados a otras clases.

De entre todas las clases que componen un programa modular en Java, debe haber una especial, a la que suele referirse como clase principal (Main class).

Los IDE suelen distinguir gráficamente el archivo de la clase principal con algún icono especial.

Ésta es la única, de entre todas, que dispone de un método principal declarado en su código (el resto no lo tienen). En la figura, Principal.java sería la clase principal. Para ejecutar un programa modular en Java, basta con ejecutar esta clase principal. A partir de ahí, la ejecución de las diferentes instrucciones sigue el flujo de control habitual, partiendo del método principal de esta clase, tal y como se ha mostrado en la figura.

Figura 1. 1. Flujo de control de un programa monolítico y uno modular en Java



Los métodos main e inicio

Dado el tipo de plantilla que se usa en estos materiales para poder dar clases con métodos, en la clase principal debe haber tanto el método main como el método inicio. Ninguno de los dos es necesario en el resto de clases.

Las clases adicionales en un programa modular, que no son la principal, normalmente tienen dos orígenes. Pueden ser clases creadas por usted mismo, o bien creadas por otros desarrolladores. En cualquiera de los dos casos, todas están declaradas y codificadas en archivos .java por separado.

1.1.3 Definición y uso de clases adicionales

Como acaba de ver, es posible dividir el código de sus programas en diferentes clases, en lugar de tenerlo en una sola. Dado que es el caso más sencillo, esta sección se centrará en la definición de clases adicionales que se comportan simplemente como repositorios de métodos y no es el caso de los tipos compuestos.

Ante todo, es necesario establecer un criterio general bajo el que decidir cuándo vale la pena dividir los métodos de un programa entre distintas clases. Normalmente, un primero criterio muy básico que debe hacer pensar en aplicar modularidad es encontrarse delante de un programa con un archivo de código fuente largo, donde hay muchos métodos. Ahora bien, lo más importante de todos es identificar métodos que realizan tareas de lo que se podría considerar una misma temática y que cree que su código puede serle de utilidad en el futuro para otros programas que genere.

Como hilo argumental para ver todo, véase un ejemplo concreto no demasiado complejo. Suponga un programa para realizar algunas operaciones típicas (máximo, mínimo y media) sobre el conjunto de notas de una clase. Para centrar el ejemplo en el aspecto importante ahora, generar un programa dividido en diferentes clases, se obviarán detalles como la interfaz (entrada desde el teclado, menús, etc.).

El programa tal y como podría plantearse directamente partiendo de una descomposición por diseño descendente, podría ser el que se muestra a continuación:

```
1 public class RegistroNotes {
2     public static void main(String[] args) {
3         RegistroNotes programa = new RegistreNotes();
4         programa.inicio();
5     }
6     public void inicio() {
7         double[] notas = {2.0, 5.5, 7.25, 3.0, 9.5, 8.25, 7.0, 7.5};
8         double max = calcularMaximo(notas);
9         double min = calcularMinimo(notas);
10        double media = calcularMedia(notas);
11        System.out.println("La nota máxima es " + max + ".");
12        System.out.println("La nota mínima es " + min + ".");
13        System.out.println("La media de las notas es " + media + ".");
14    }
15    public double calcularMaximo(double[] array) {
16        double max = array[0];
17        for (int i = 1; i < array.length; i++) {
18            if (max < array[i]) {
19                max = array[i];
20            }
21        }
22        return max;
23    }
24    public double calcularMinim(double[] array) {
25        double min = array[0];
26        for (int i = 1; i < array.length; i++) {
27            if (min > array[i]) {
28                min = array[i];
29            }
30        }
31        return min;
32    }
33    public double calcularMedia(double[] array) {
34        double suma = 0;
35        for (int i = 0; i < array.length; i++) {
36            suma = suma + array[i];
37        }
38        return suma/array.length;
39    }
40 }
```

Aunque no sea un programa muy largo, se aplicará modularidad. De entrada, una característica que sí se puede identificar es el hecho de que existen tres métodos de temática similar: realizar operaciones típicas basadas en recorridos sobre arrays de reales (calcularMaximo, calcularMinimo, calcularMedia). Este tipo

de operaciones son muy generales y seguro que se suelen usar en diferentes programas. Por tanto, puede tener sentido declararlos en una clase aparte que sirva como un repertorio general de métodos para trabajar con arrays de reales.

Definición de clases adicionales

Para dividir un programa en clases diferentes, basta con crear tantos archivos como clases se quiere, cada uno con su nombre y definición propia, tal y como ha hecho hasta ahora. De entre todas ellas alguna deberá ser la clase principal (en este caso, que contenga al menos los métodos main e inicio). El resto contendrá los métodos que se quiera distribuir.

Si en el ejemplo queremos que `RegistreNotes` sea la clase principal y declarar los tres métodos para trabajar con arrays de reales en una nueva clase llamada `CalculsArrayReals`, habría que crear un nuevo archivo con el siguiente contenido:

```
1 public class CalculsArrayReals {  
2     public double calcularMaximo(double[] array) {  
3         double max = array[0];  
4         for (int i = 1; i < array.length; i++) {  
5             if (max < array[i]) {  
6                 max = array[i];  
7             }  
8         }  
9         return max;  
10    }  
11    public double calcularMinim(double[] array) {  
12        double min = array[0];  
13        for (int i = 1; i < array.length; i++) {  
14            if (min > array[i]) {  
15                min = array[i];  
16            }  
17        }  
18        return min;  
19    }  
20    public double calcularMedia(double[] array) {  
21        double suma = 0;  
22        for (int i = 0; i < array.length; i++) {  
23            suma = suma + array[i];  
24        }  
25        return suma/array.length;  
26    }  
27 }
```

Fuera de la falta de un método principal en las clases adicionales, todas las clases son exactamente iguales a nivel de sintaxis. Todo lo que ha aprendido se puede aplicar para cualquiera de ellas (definición y uso de constantes, variables globales, ámbito de variables, etc.).

Uso de clases adicionales

Una vez declarados en una clase aparte, los métodos pueden ser eliminados de la clase original (`RegistreNotes`), ya que de lo contrario tendría código repetido. Precisamente, en un programa modular, lo que sucede es que desde una clase se invocan métodos declarados en una clase diferente. Pero cada método sólo está escrito una sola vez entre todas las clases del programa.

La invocación de métodos que no están escritos en el mismo archivo es diferente del caso que ha visto hasta ahora, donde todas están escritas dentro del archivo de una misma clase. Ya no se puede llevar a cabo simplemente escribiendo el nombre del método (junto con sus parámetros, si procede). Primero es necesario un paso previo de inicialización, a partir del cual se permite la invocación de los métodos externos. Al llevar a cabo este proceso, se asigna un identificador a partir del cual es posible invocar los métodos de esa clase, usándolo como prefijo en la invocación. Sin ese paso previo, es imposible invocar métodos escritos en otras clases.

La sintaxis para ello es la siguiente:

```
1 NombreClase identificador = new NomClase();  
2 identificador.nombreMétodo(parámetros);
```

El comportamiento de la invocación del método es idéntica a cuando se hace sobre un método escrito en la misma clase (uso de parámetros, evaluación de acuerdo a su valor de regreso). Sólo cambia la sintaxis, ya que se requiere este prefijo producto de la inicialización. Hay que decir que, dadas diversas invocaciones a métodos de una misma clase externa, sólo es necesario realizar la inicialización una única vez. El ámbito y validez del identificador es el mismo que una variable.

En el caso del ejemplo, el código de la clase RegistroNotes cambiaría ahora por siguiente:

```
1 public class RegistroNotes {  
2     public static void main(String[] args) {  
3         RegistroNotes programa = new RegistreNotes();  
4         programa.inicio();  
5     }  
6     public void inicio() {  
7         double[] notas = {2.0, 5.5, 7.25, 3.0, 9.5, 8.25, 7.0, 7.5};  
8         //Para llamar a los métodos es necesario inicializar la clase que los contiene  
9         CalculsArrayReals calculador = new CalculsArrayReals();  
10        //Una vez hecho, hay que llamarlos usando como prefijo el identificador  
11        double max = calculador.calcularMaximo(notas);  
12        double min = calculador.calcularMinimo(notas);  
13        double media = calculador.calcularMedia(notas);  
14        System.out.println("La nota máxima es " + max + ".");  
15        System.out.println("La nota mínima es " + min + ".");  
16        System.out.println("La media de las notas es " + media + ".");  
17    }  
18 }
```

En nomenclatura Java, el proceso de inicialización se llama formalmente instanciación de la clase.

Reutilización de módulos

Como ya se ha dicho, el motivo principal para dividir un programa en módulos es poder reusarlos en otros programas donde es necesario realizar tareas iguales. Por ejemplo, suponga que se le pide hacer un programa para llevar a cabo un registro de temperaturas. En éste, se quiere mostrar la diferencia máxima entre las temperaturas registradas (cuánto hay entre el valor máximo y el mínimo). Analizando este problema, ya puede ver a primera vista que hay que trabajar con un array de valores reales con los que realizar operaciones típicas sobre listas de valores: calcular el máximo y el mínimo. Precisamente algunas de las tareas que acaba de resolver para gestionar notas.

Hasta ahora, en un caso como éste, simplemente podría tomar el programa en el que ya se ha resuelto este problema y hacer “copiar y enganchar” del código al nuevo programa. Mediante la modularidad, no es necesario hacer esto. Basta con incorporar la clase en la que hay existen los métodos que le interesa usar en el nuevo programa, e invocarlos directamente. Por tanto, para este nuevo programa podría incorporar una copia de la clase `CalculsArrayReals` y hacer la siguiente clase principal:

```
1 public class RegistroTemperaturas {  
2     public static void main(String[] args) {  
3         RegistroTemperaturas programa = new RegistroTemperaturas();  
4         programa.inicio();  
5     }  
6     public void inicio() {  
7         double[] tiempo = {20.0, 21.5, 19.0, 18.5, 17.5, 19.0, 22.25, 21.75};  
8         //Para llamar a los métodos es necesario inicializar la clase que los contiene  
9         CalculsArrayReals calculador = new CalculsArrayReals();  
10        //Una vez hecho, hay que llamarlos usando como prefijo el identificador  
11        double max = calculador.calcularMaximo(tiempo);  
12        double min = calculador.calcularMinimo(tiempo);  
13        double dif = max - min;  
14        System.out.println("La diferencia de temperatura máxima"           + dif + ".");  
15    }  
16 }
```

Reto 1. Haga un programa que, dados dos arrays de valores reales, le diga cuál de ambos tiene el valor medio más alto. Aproveche el principio de modularidad para hacerlo, de modo que su clase principal sólo tenga definidos los métodos `main` y `inicio`.

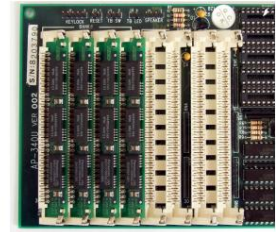
1.1.4 Ventajas de la modularidad usando clases adicionales

Una vez visto que existe la posibilidad de crear programas modulares distribuyendo el su código en diferentes clases, vale la pena plantearse cuáles son las ventajas que aporta. Ante todo, cabe decir que esta aproximación sólo suele tener sentido en programas de cierta complejidad, o donde existe un conjunto extenso de métodos con funcionalidades genéricas, que claramente pueden ser de gran utilidad en la creación de programas futuros. De lo contrario, cuando hay muy pocos métodos, no es necesario aplicarla estrictamente.

La principal ventaja de la modularidad es poder reutilizar directamente código ya generado anteriormente en nuevos programas, sin tener que volver a escribirlo o copiarlo por clases diferentes cada vez que se quiere utilizar en distintos programas. Este código siempre estará en la misma clase dentro de cualquier programa, cosa que le hace más fácil de localizar y gestionar. Por otra parte, si alguna vez en el futuro se hacen enmiendas a alguno de los métodos de esta clase (por ejemplo, arreglar un error que se ha detectado al cabo de un tiempo), basta con reemplazar la versión antigua de la clase por la nueva. Esto se hace simplemente mirando si hay un archivo y, si es así, sobrescribirlo. Compare esto con el trabajo que sería buscar los pedazos de código enmendados y reemplazarlos usando un editor (abrir con un editor el código fuente de los vuestros programas, buscar si existen los métodos modificados y sobrescribirlos con la nueva versión en su caso).

Otra ventaja interesante de generar un programa de manera modular es facilitar el trabajo en equipo, una opción indispensable dentro del mundo del desarrollo de software actual, donde las aplicaciones son complejas y se necesitan diferentes especialistas que deben coordinarse. Distribuyendo el código en diferentes clases, basta con establecer el esqueleto de todas ellas (la declaración de los métodos), y entonces se puede repartir el trabajo de forma que cada desarrollador realice el código de unas pocas clases de entre todas. Todos pueden trabajar en paralelo sin problemas, lo que es mucho más complicado si todo está dentro del mismo archivo.

Evidentemente, la modularidad también tiene algún inconveniente menor del que vale la pena ser consciente de ello. Dada una clase creada con anterioridad y que se quiere reutilizar, quizás de los diversos métodos contenidos en realidad sólo desea utilizar unos pocos. Por tanto, el programa a menudo contiene clases con pedazos de código que nunca se usan. En el caso del ejemplo del registro de temperaturas, el método para calcular el promedio nunca se usa, pero estará definido igualmente en una parte del código fuente. Ahora bien, se considera que las ventajas de la modularidad son tan superiores a este inconveniente que es un mal menor sobre el que no hace falta pensar demasiado si desea diseñar sus programas.



Las clases son como los módulos de memoria de un ordenador. Se puede sacar una para poner otra mejorada. Fuente: Appaloosa

Los programas modulares permiten facilitar la tarea de desarrollar software complejo, de modo que sea más fácil generar y mantener el código, especialmente cuando se trabaja en equipo.

1.2 Bibliotecas de clases: packages

Normalmente, para aplicar el principio de modularidad dentro de un programa es suficiente con organizar los métodos dentro de clases diferenciadas, de acuerdo a algún criterio de ordenación o de reutilización en el futuro. Ahora bien, para casos en los que se quiere crear un programa con un cierto grado de complejidad, puede suceder que el número de clases resultantes también sea bastante grande. En un caso como éste, también podría ser conveniente disponer de un mecanismo que permita organizar conjuntos de clases dentro de un programa, al igual que una clase organiza un conjunto de métodos. Esto permite aplicar modularidad a conjuntos de clases en bloque, en lugar de hacerlo a nivel individual, así como hacerlas más fáciles de localizar y gestionar.

Una biblioteca de clases, o package en Java, es un conjunto de clases vinculadas entre sí de acuerdo a algún criterio temático o de organización de su código.

A menudo, los criterios para decidir cómo dividir las clases de un programa en conjuntos de packages diferentes son bastante subjetivos. En este caso, lo que debe tener en cuenta es que se trata de una herramienta de la que dispone libremente para organizar los diferentes archivos de un programa a su gusto, al igual que se puede

organizar un conjunto de fotografías referentes a varios viajes de muchas formas usando una estructura de carpetas.

En este sentido, cabe decir que, si bien la mayoría de los ejemplos con los que ha trabajado hasta ahora (y con los que se seguirá trabajando) son programas con un número de métodos demasiado limitado como para justificar de forma clara el uso de packages, se trata de un elemento suficientemente importante de Java como para hacer necesario conocer su funcionamiento.

1.2.1 Definición de packages

Para asignar un conjunto de clases a un package, primero debe elegir un identificador, que servirá como el nombre del mismo. Como siempre, este identificador debería ser autoexplicativo respecto a la función de las clases que va a contener. Una vez elegido, para que una clase pertenezca, basta con que en la primera línea de su archivo con el código fuente se escriba la sentencia mostrada a continuación. Su aparición debe ser estrictamente siempre en la primera línea de texto, incluso antes de la declaración de clase (public class...).

```
1 package identificadorPackage;
```

URL

Iniciales de "Localizador Uniforme de Recursos" (Uniform Resource Locator, en inglés). Es la cadena de caracteres que informa en el navegador al que desea conectarse.

Es preciso notar, pues, que el concepto package en Java no tiene una entidad propia diferenciada. Al contrario que las clases, no es ningún archivo concreto a editar. Será dado implícitamente por todas las clases que se declaren como parte de él en su código fuente.

Como convención de código, el identificador de un package debería seguir un formato especial, al igual que existen convenciones para los nombres de las clases, métodos y variables. El programa no deja de funcionar si no se hace así, pero vale la pena seguir esta convención, puesto que es el formato usado en la inmensa mayoría de programas en Java. Al hacerlo, su código aporta la imagen de estar hecho por programadores serios y cuidadosos. En este caso, se trata de que los nombres de los packages se escriban siempre todos en minúscula y separando palabras diferentes por un punto, por lo que tienen un aspecto similar al URL escrito en la barra de dirección de un navegador web. Por ejemplo:

```
1 package unidad5.apartado1.ejemplo1;
```

Los creadores del lenguaje Java desaconsejan utilizar el package por defecto.

Estrictamente, todas las clases de Java pertenecen a algún package. En caso de no incluir ninguna sentencia `importe`, se considerará que aquella clase pertenece a un package especial llamado por defecto (default package). No es posible crear una clase que "no pertenezca a ningún package". Por tanto, todos los programas que ha creado hasta el momento se han compuesto de clases que formaban parte del package por defecto, a pesar de no usar la sentencia `import`.

La forma en que tiene Java de ordenar las clases dentro de un programa en packages (incluido el package por defecto) comporta dos restricciones que debe tener siempre en cuenta:

- Dada una clase, ésta únicamente puede pertenecer a un package. No es posible utilizar más de una vez la sentencia package en el código fuente de una clase.
- Dado un package, en su interior nunca puede haber dos clases con el mismo nombre. Por regla general, puede considerarse que esta regla se aplica ignorando mayúsculas y minúsculas.

Packages e IDE

Para facilitar el trabajo del desarrollador, la mayoría de IDE actuales, en realidad, sí tratan a los packages como una entidad especial dentro de sus proyectos. Así pues, antes de poder crear ninguna clase contenida dentro de un package, es necesario crearlo explícitamente de forma individual. Una vez hecho, es posible añadir clases, de forma similar a cómo se gestionarían archivos dentro de una carpeta. Cuando se crea una nueva clase dentro de cada package, el IDE ya se encarga de incluir automáticamente la sentencia `import` al inicio del código fuente. Adicionalmente, mientras no se cree ningún package con un nombre concreto, también suelen mostrar explícitamente el package por defecto, donde van incluyendo las nuevas clases que cree.

Sin embargo, tenga en cuenta que esta característica sólo es un mecanismo de la interfaz de usuario de los IDE para hacer su gestión más sencilla.

Reto 2. Genere un proyecto en su IDE con un package llamado `unidad5.apartado1.reto2`. Haga que dentro de este package se incluyan las clases del ejemplo del registro de las notas y ejecútelo, comprobando que funciona correctamente.

1.2.2 Criterios para crear bibliotecas de clases

Normalmente, los criterios usados para dividir conjuntos de clases en bibliotecas distintas suelen ser:

- Un programa completo, una jerarquía de packages.
- Muchos programas pequeños, una jerarquía de packages.
- Un conjunto de clases sin un programa principal, un package.

Dado que el uso de bibliotecas no es más que un mecanismo para ordenar las clases de un programa, inicialmente se puede realizar una analogía con la ordenación de un conjunto de fotografías de diferentes viajes dentro de carpetas. Cada viaje equivale a un programa distinto en esta analogía.

1. Un programa completo, una jerarquía de packages.

Inicialmente, podría tener una carpeta para cada viaje, en cada una de las cuales están todas las fotos asociadas directamente. Esto sería equivalente a usar cada biblioteca para encapsular un programa completo, compuesto por la

su clase principal y una serie de clases adicionales. Así pues, se asignaría el mismo identificador a todas las clases que conforman el programa.

Bajo este criterio, las clases de los ejemplos, al ser programas diferentes, podrían ir dentro de los diferentes packages `unidad5.apartado1.ejemplo1`, `unidad5.apartado1.reto1`, etc.

Sin embargo, partiendo de este criterio podría considerar poner algunas de las fotos de un viaje, a su vez, en carpetas según el día, o el monumento visitado, por lo que hay una carpeta con fotos “generales” y algunas subcarpetas que agrupan visitas concretas. De hecho, éste es el caso más típico cuando se usan bibliotecas en Java para encapsular aplicaciones. Hay una biblioteca donde se guarda la clase principal y después de otras donde se agrupan las clases adicionales por tipos de función. En este caso, lo que se hace es extender el identificador base con nombres que indiquen el tipo de clases adicionales.

Tenga en cuenta que todos estos packages son diferentes a nivel de estructura del programa. Pero el nombre suele elegirse estratégicamente para que a partir de él se deduzca ya a simple vista que existe una relación jerárquica entre las clases. Las clases del nivel más alto de la jerarquía hacen uso de las de niveles más bajos.\\Más adelante, si la aplicación realiza otras acciones de otras temáticas, se podrían añadir un nuevo package. Por ejemplo, para tratar la lectura de datos por el teclado se podría generar el package `unidad5.apartado1.ejemplo1.entrada`, en el que irían las clases vinculadas a la entrada de datos.

Por ejemplo, en el caso del ejemplo mostrado, podríamos tener dos packages:

- `unidad5.apartado1.ejemplo1`, que engloba a la clase principal.
- `unidad5.apartado1.ejemplo1.arrays`, que engloba todas las clases para tratar los datos. Ahora mismo sólo contendría `CalculsArrayReals`, pero más adelante podría incluir otras clases para extraer información adicional de los datos del programa (predecir temperaturas en el futuro, realizar histogramas o gráficos, etc.)

2. Muchos programas pequeños, una jerarquía de packages

Otra opción posible es englobar a más de un programa en un único package. Esto suele hacerse cuando se quiere gestionar fácilmente un conjunto de programas muy simples, normalmente de una sola clase, que a menudo usan las mismas clases adicionales. Siguiendo la analogía, si ha realizado varios viajes muy breves, o de paso, a una misma ciudad, por lo que de cada viaje sólo tiene 3 o 4 fotos, no vale la pena diferenciarlos, sale más a cuenta ponerlas todas mezcladas en la misma carpeta, llamada de acuerdo a la ciudad.

El objetivo principal es evitar la explosión en el número de packages, que al final terminan teniendo una o dos clases. Ahora bien, esto tiene una particularidad, y es que en un mismo package hay más de una clase principal mezclada.

De hecho, un package no tiene límite alguno en el número de clases principales que puede contener. Basta con que, en su entorno de desarrollo, elija de forma adecuada cuál es la que desea que se ejecute de entre todas las existentes. Java se encarga del resto.

Si se para a pensar un momento, este es el caso precisamente de la mayoría de ejemplos o retos propuestos, al ser programas con muy pocos archivos y que a menudo compartirán algunas clases adicionales, por lo que es lo que se usará a partir de ahora. Por ejemplo, organizando las clases según los packages llamados `unidad5.apartado1.ejemplos` y `unidad5.apartado1.retos`.

3. Un conjunto de clases sin programa principal, un package

Por último, puede haber clases que se consideran de propósito general, útiles en muchos programas diferentes. Esto puede suceder si desea dejar la puerta abierta a reutilizar clases que está implementando en otro nuevo programa, más adelante. O sea, fotografías que aplican a más de un viaje.

Normalmente, lo que haría es crear una carpeta aparte y ponérselas, en lugar de tenerlas en la de un viaje concreto. En un caso como éste, la introducción del concepto de biblioteca de clases plantea una nueva posibilidad a la hora de crear código. Concretamente, el caso inverso del anterior, el desarrollo de repertorios de clases que no conforman propiamente un programa, puesto que ninguna de ellas es la principal.

En el ejemplo visto, éste es el caso de la clase `CalculsArrayReals`. En un caso como éste, las clases de propósito general suelen crearse en una biblioteca aparte. Seguro que una serie de clases para realizar operaciones con arrays se usarán más allá de la unidad 5. Por tanto, no tiene sentido definir las en un package llamado `unidad5.apartado1.ejemplos.arrays`, sino usando un nombre que no esté asociado a esta jerarquía. En ese caso, por ejemplo, podría ser el package `utilitats.arrays`.

Después de cierta experiencia con Java al final es evidente qué tipo de operaciones típicas se acaban repitiendo en todos los programas.

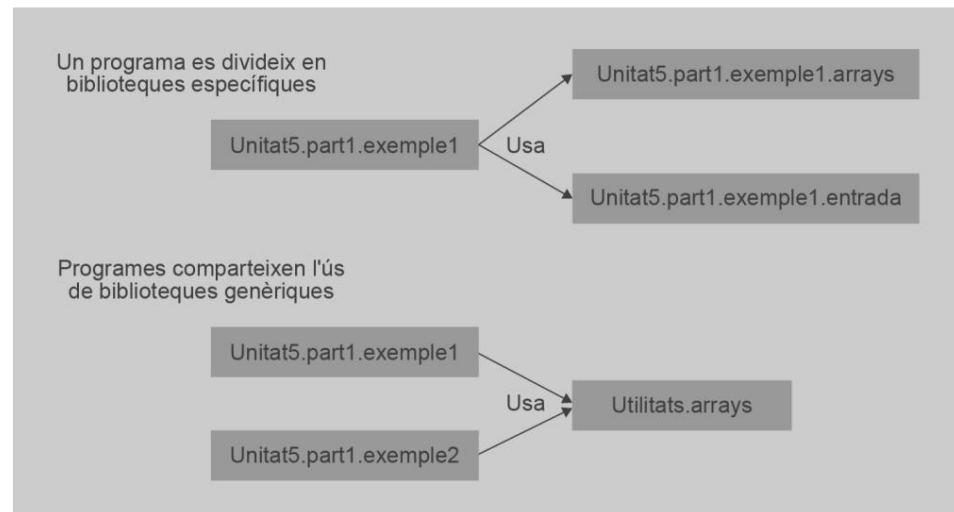
Más aún, se puede plantear el desarrollo de un conjunto de clases en una biblioteca que no sean producto parcial de la generación de un programa, que posteriormente han resultado ser reutilizadas, sino que ya desde el punto de partida se han creado como herramientas de soporte para programas futuros. Por tanto, no es inconveniente tener packages sin ninguna clase principal (de hecho, es el caso más habitual). Una vez que se identifica que un conjunto de clases se usa en muchos programas diferentes, lo mejor es hacer que formen parte de su propia biblioteca independiente, fuera de la jerarquía del nombre de ninguna aplicación concreta.

La figura 1.2 muestra un esquema que contrasta este criterio para gestionar clases adicionales con el que crear una relación jerárquica entre el package con la clase principal.

Sin embargo, en última instancia debe tener en cuenta que no hay una respuesta totalmente correcta a la hora de englobar clases dentro de bibliotecas.

Sólo se han enumerado unos criterios que suelen utilizarse dentro de los programas profesionales realizados en Java. Pero, en el fondo, los packages sólo son una herramienta para organizar las clases de manera coherente para que le resulte más cómoda a vosotros.

Figura 1. 2. Criterios de estructuración de programas en bibliotecas



1.2.3 Uso de clases de otros packages

Para referirnos a una clase simplemente se debía llamar su nombre (o el nombre de su archivo). Dado que toda clase siempre pertenece a una biblioteca, y dentro de bibliotecas diferentes pueden haber clases con igual nombre, puede ser interesante disponer de una herramienta para decir exactamente de qué clase se está hablando en cada caso.

El nombre calificado de una clase es la combinación del identificador de su package junto a su nombre, separados por un punto.

Ejemplos de nombres calificados, partiendo de los ejemplos anteriores
 oros, serían `unidad5.apartado1.ejemplos.RegistroNotas` o
`utilidades.arrays.CalculsArrays`. Con un nombre calificado queda totalmente
 claro a qué package pertenece la clase a la que se está refiriendo en un momento
 dado, independientemente del contexto.

El nombre calificado de las clases es útil ya que desde el código de una clase que pertenece a un package concreto, sólo se pueden usar directamente clases con las que comparta el mismo nombre de package. Si se usa una clase de uno diferente, el compilador dará un error, diciendo que no la reconoce. Esto pasaría si, al organizar las clases `RegistroNotes` y `CalculsArrayReals` en bibliotecas diferentes, desde la primera se intenta utilizar directamente la segunda. Ya no funciona. Hay que establecer alguno mecanismo para indicar exactamente qué clase se está usando. Java ofrece tres formas, más o menos relacionadas.

1. Inicialización usando el nombre calificado

Por un lado, para el caso de clases de otros packages, se puede utilizar su nombre calificado dentro del código fuente para referirse a su inicialización.

```
1 package unidad5.apartado1.ejemplos; 2 public class
RegistroNotes {
3     ...
4     //Uso del nombre calificado para acceder a una clase de otro package
5     utilitats.arrays.CalculsArrayReals calculador = new utilitats.arrays.
        CalculsArrayReals();
6     ...
7 }
```

```
1 package utilidades.arrays; 2 public class
CalculsArrayReals { //Código
3
4     ...
5 }
```

2. Importación explícita

Otra forma, mucho más cómoda si una clase se usa varias veces dentro del código, es importarla previamente a la cabecera del archivo de código fuente, entre la declaración del package y la declaración public class.... Para realizar esta importación, es necesario especificar el nombre calificado de la clase que se quiere usar. La sintaxis general es:

```
1 importe nombreCalificado;
```

En caso de querer usar más de una clase, ya sea de la misma biblioteca o de diferentes, será necesario poner tantas sentencias importe como corresponda. Por ejemplo, si utiliza cinco clases que no pertenecen al mismo package que la clase que está creando, habrá que incluir cinco sentencias importe por separado. En este proceso, las relaciones de jerarquía entre los identificadores del package no importan, es necesario poner cada clase con su package explícitamente, una por una.

Siguiendo el ejemplo anterior de división de las clases CalculsArrayReals y RegistroNotes en packages diferentes, para poder usar la primera clase en la segunda de esta forma, habría que hacerlo así:

```
1 package unidad5.apartado1.ejemplos; 2 //Hay que importar
la clase "CalculsArrayReals", ya que es de otro package 3 importe utilidades.arrays.CalculsArrayReals; 4 public class
RegistroNotes { //Código
5
6     ...
7 }
```

```
1 package utilidades.arrays; 2 //No se usa
ninguna clase fuera de este package, no importa nada 3 public class CalculsArrayReals { //Código
4
5     ...
6 }
```

3. Importación general

Por último, existe un tercer mecanismo que permite importar automáticamente todas las clases dentro de una biblioteca. Éste se utiliza a menudo cuando se quieren usar muchas clases del mismo package e importarlas una a una se hace pesado. Se trata de usar

un asterisco, “*”, en lugar del nombre de la clase en el nombre calificado. Éste hace de comodín y equivale a decir “absolutamente todas las clases del package”.

```
1 package unidad5.apartado1.ejemplos; 2 //Se importarían todas
las clases del package utilitats.arrays 3 importe utilidades.arrays.*; 4 public class RegistroNotes { //Código

5
6     ...
7 }
```

Por lo general, es aconsejable usar la importación explícita, ya que así al inicio de su código fuente siempre dispondrá de una lista detallada de qué clases de otras bibliotecas está usando.

Reto 3. Modifique el ejemplo del registro de temperaturas de forma que la clase RegistroTemperaturas pertenezca al package unidad5.apartado1.reto3 y la clase CalculsArrayReals en el package utilitats.arrays. Asegúrese de que el programa se ejecuta correctamente.

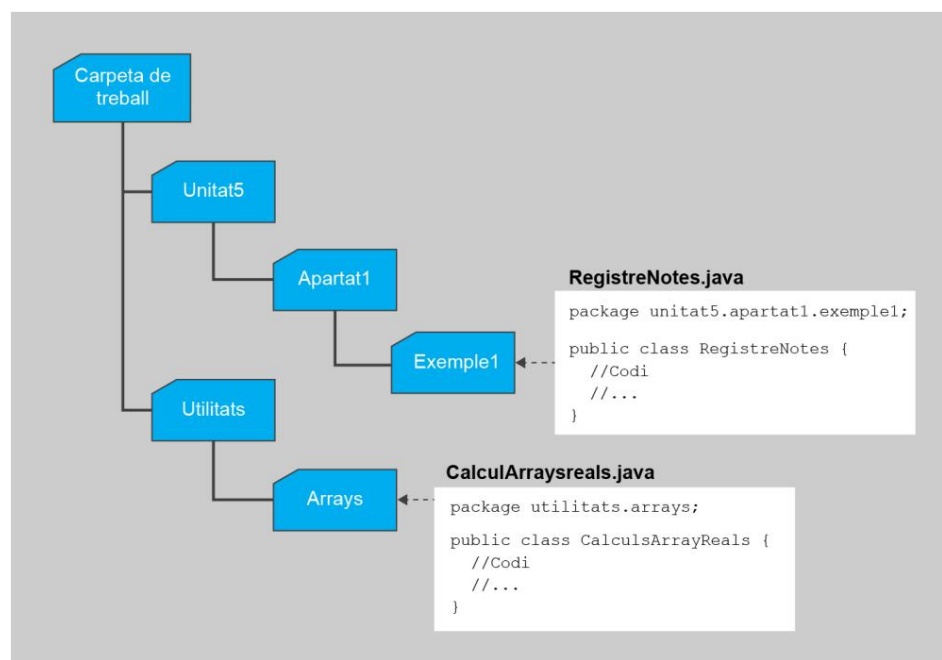
1.2.4 Estructura de los archivos dentro de los packages

Cada IDE guarda las clases en una carpeta considerada "de trabajo" diferente, pero siempre estará dentro del proyecto creado. A menudo se llama src.

El identificador elegido como nombre de un package en Java indica cómo se organizan las clases contenidas dentro del sistema de archivos de su ordenador.

Partiendo de la carpeta considerada de trabajo, todas las clases que forman parte del package deben estar ubicadas dentro de una jerarquía de carpetas donde cada carpeta tiene el nombre de cada uno de los textos separados por puntos que conforman el nombre completo del package. En la última carpeta es donde estarán ubicados realmente los archivos .java.

Figura 1. 3. Despliegue de clases en carpetas de acuerdo al nombre del package.



Organizar las clases en esta estructura de carpetas según su package sirve para garantizar dos cosas importantes. Por un lado, que las clases de diferentes packages están ordenadas en carpetas diferentes, fácilmente identificables, por lo que no se mezclan todas. Por otro lado, también permite que dentro de dos packages diferentes pueda haber clases con el mismo nombre sin que implique ninguna colisión en el sistema de archivos (que los archivos .java se sobrescriban).

Si se sigue con el ejemplo de las clases vinculadas al registro de notas, partiendo de criterio de hacer que `CalculsArrayReals` pertenezca a una biblioteca común, las clases estarían distribuidas de la forma que muestra la figura 1.3.

Si en un momento dado se quieren usar las clases del package `utilitats.arrays` en otro programa, no basta con copiar simplemente los archivos .java en el directorio de la nueva aplicación. Es necesario que éstas se copien manteniendo su estructura de carpetas. Por tanto, habría que copiar la estructura existente partiendo de la carpeta `utilidades`.

Packages e IDE (II)

Los IDE actuales también se encargan de gestionar la organización de la estructura de directorios de acuerdo al nombre de los packages de forma automática, una vez se ha creado un explícitamente y se van añadiendo clases. Por tanto, si usa uno, no debe preocuparse de este hecho. De todas formas, tenga muy en cuenta si copia clases entre proyectos, ya que hay que seguir las reglas recién explicadas.

1.3 La API del lenguaje Java

Para realizar programas modulares, no siempre es necesario crear todos y cada uno de los módulos que forman parte del programa. Por suerte, a menudo los lenguajes de programación incorporan bibliotecas de módulos auxiliares que ya han sido completamente desarrollados por los creadores del lenguaje u otros colaboradores. Estos módulos auxiliares extienden el lenguaje de programación, ofreciendo funcionalidades que van más allá de lo que permite su sintaxis estrictamente o incorporando la opción de ejecutar bloques de código que resuelven tareas que se consideran de propósito general y que pueden ser de utilidad en una amplia gama de programas.

En el caso del lenguaje Java, su kit de desarrollo (JDK) incorpora un cuantioso repositorio de clases, organizadas por distintos packages de acuerdo a su temática, que pueden ser accedidas libremente al realizar cualquier programa. Este repositorio es el llamado API de Java. Ahora bien, el objetivo de esta sección no es conocer todas las clases disponibles, sino simplemente que tenga un marco general sobre cómo usar una que sea sencilla, una vez ya es conocida su existencia y que puede ser útil para hacer un programa.

Para la versión 1.6, el listado de todas las clases de Java, junto con su documentación, se puede encontrar en la dirección de Oracle de las Direcciones de interés de la web.

Si observa la documentación de la API de Java, de la que la figura 1.4 muestra una idea de su estructura, verá que ofrece un enorme repertorio de clases que contienen métodos con funcionalidades muy diversas, lo que, en ocasiones, hace muy complicado encontrar de entre todas la clase que contiene los métodos ideales para cada caso, si es que existe. De hecho, puede considerar que intentar averiguar

Figura 1. 4. La API de Java se compone realmente de una gran cantidad de clases



- La ventaja de esta aproximación es que, como estas clases ya están creadas e incorporadas como parte del lenguaje, si encuentra una que ya haga el trabajo requerido, se ahorrará tener que escribir partes de código fuente del programa.

Para poder usar cualquier clase de entre las definidas en la API de Java es necesario seguir exactamente los mismos pasos que para una clase realizada por vosotros, pero

dentro de un package distinto: importarla al inicio del código fuente e inicializarla. Una vez realizado, la sintaxis para invocar sus métodos es exactamente la misma. Este proceso sólo tiene una particularidad muy especial, y es que no es necesario disponer del código fuente de la clase en cuestión. Al estar incorporada en el JDK, Java ya sabe localizar su código automáticamente sin tener que incorporarlo explícitamente.

Para repasar este proceso, se utilizará como ejemplo sencillo partiendo desde cero. Suponga que debe realizar un programa que debe generar dos valores reales al azar, entre 0 y 100, y que los muestre por pantalla. De entrada, resolver este problema supondría crear dentro del código fuente algún método que generara estos valores reales aleatorios. Por suerte, investigando un poco fuentes de documentación, no es complicado encontrar que existe una clase llamada Random, ya incorporada en la API de Java, la cual permite realizar este tipo de tareas. Por lo tanto, estudiando cómo usar sus métodos puede ahorrarle trabajo de codificación. Buscándola en la documentación de la API de Java, se ve que forma parte del package java.util.

Sin saber aún los detalles de esta clase, se puede hacer el esqueleto de la clase principal del programa, con la correspondiente importación. Tenga cuidado con que este ejemplo incluye la clase dentro de un package.

```
1 package unidad5.apartado1.ejemplos; 2 //Importación de la clase, al
estar en otro package 3 importe java.util.Random; 4 public class RealsAleatorios { public static void main(String[]
args) {
5
6     RealsAleatorios programa = new RealsAleatorios();
7     programa.inicio();
8
9 } public void inicio() {
10     //Hay que generar un valor al azar
11     //Hay que generar el otro
12     //Mostrarlos
13 }
14 }
```

Su programa sólo tiene que estar compuesto por esta clase. Al formar parte de la API de Java, el uso de la clase Random no implica la incorporación de ningún otro archivo de código fuente. Una vez ya está listo para usarla, habrá que mirar qué métodos ofrece para generar reales al azar. Por eso es necesario consultar la documentación, como muestra la figura 1.5. En el apartado de resumen de métodos (Method Summary) existe una lista de los métodos con una descripción general. Entre todos, se puede encontrar uno que parece generar valores reales al azar, llamado `nextDouble()`. Si se quiere ver con más concreción qué hace el método, la documentación proporciona una descripción más detallada (Method Detail), en la que siempre vale la pena echar un vistazo. Sin embargo, de la información obtenida se puede ver que en realidad este método genera valores al azar entre 0.0 y 1.0. Por tanto, si se quieren crear valores entre 0 y 100, no hace todo el trabajo. Habrá que hacer una multiplicación de su resultado por 100. No siempre encontrará métodos que hacen exactamente lo que desea, pero basta con ahorrarle los trabajos más complicados.

Figura 1. 5. Estudiando los métodos ofrecidos por una clase

java.sql
java.text
java.text.spi
java.util
java.util.concurrent
java.util.concurrent.atomic
java.util.concurrent.locks
java.util.jar
java.util.logging
java.util.prefs
java.util.regex
java.util.spi
java.util.zip
javax.accessibility
javax.activation

LinkedList
ListResourceBundle
Locale
Observable
PriorityQueue
Properties
PropertyPermission
PropertyResourceBundle
Random
ResourceBundle
ResourceBundle.Control
Scanner
ServiceLoader
SimpleTimeZone
Stack
StringTokenizer
Timer
TimerTask

Method Summary

protected	next (int bits)	Generates the next pseudorandom number.
int	nextInt ()	Returns the next pseudorandom, uniformly distributed <code>int</code> value from this random number generator's sequence.
boolean	nextBoolean ()	Returns the next pseudorandom, uniformly distributed <code>boolean</code> value from this random number generator's sequence.
void	nextBytes (byte[] bytes)	Generates random bytes and places them into a user-supplied byte array.
double	nextDouble ()	Returns the next pseudorandom, uniformly distributed <code>double</code> value between 0.0 and 1.0 from this random number generator's sequence.
float	nextFloat ()	Returns the next pseudorandom, uniformly distributed <code>float</code> value between 0.0 and 1.0 from this random number generator's sequence.
double	nextGaussian ()	Returns the next pseudorandom, Gaussian ("normally") distributed <code>double</code> value with mean 0.0 and standard deviation 1.0 from this random number generator's sequence.
int	nextInt ()	Returns the next pseudorandom, uniformly distributed <code>int</code> value from this random number generator's sequence.
int	nextInt (int n)	Returns a pseudorandom, uniformly distributed <code>int</code> value between 0 (inclusive) and the specified value (exclusive), drawn from this random number generator's sequence.
long	nextLong ()	Returns the next pseudorandom, uniformly distributed <code>long</code> value from this random number generator's sequence.
void	setSeed (long seed)	Sets the seed of this random number generator using a single <code>long</code> seed.

Method Detail

• • •

nextDouble

```
public double nextDouble ()
```

Returns the next pseudorandom, uniformly distributed `double` value between 0.0 and 1.0 from this random number generator's sequence.

The general contract of `nextDouble` is that one `double` value, chosen (approximately) uniformly from the range 0.04 (inclusive) to 1.04 (exclusive), is pseudorandomly generated and returned.

El código para generar dos valores reales al azar quedaría como muestra el siguiente código.

A nivel de sintaxis, todo es igual que si fuera una clase adicional generada por vosotros mismos. Compruebe que, efectivamente, para diferentes ejecuciones los valores mostrados por pantalla varían al azar.

```
1 ...
2 public void inicio() {
3     //Iniciación
4     Random rnd = new Random();
5     //Uso
6     double valorA = rnd.nextDouble() * 100;
7     double valorB = rnd.nextDouble() * 100;
8     System.out.println("Se han generado los valores" + valorA + " y " + valor B);
9 }
10 ...
```

Una de las características esenciales de Java es la su modularidad inherente.

Si hace un poco de memoria, verá que el proceso justo descrito ya lo ha usado anteriormente en varias ocasiones: cada vez que necesitaba leer datos para teclado y ha usado Scanner. Si ve la documentación de la API de Java, encontrará esta clase en el mismo package `java.util`. De hecho, `String` también es una clase incorporada dentro de la API de Java, sólo que al formar parte del package `java.lang`, es un caso especial y no es necesario importarla explícitamente.

Así pues, aunque no han sido conscientes de ello, muchos de sus programas hechos ¡hasta ahora ya eran, en cierto modo, modulares!

1.3.2 Inicialización con parámetros

Algunas clases de Java tienen una particularidad, y es que a la hora de inicializarlas es necesario especificar un conjunto de información adicional en forma de parámetros. Éste

caso se tratará aquí sólo de manera superficial ya nivel de uso dentro del contexto de las clases ofrecidas dentro de la API de Java, ya que es bastante habitual, pero no a nivel de cómo poder hacer que las clases generadas por ustedes sean capaces de aceptarlas, ya que es más complejo. El objetivo sólo es que disponga del trasfondo básico para poder trabajar con algunas de las clases más sencillas de la API.

Hasta el momento, se ha dicho que para inicializar una clase para poder invocar los métodos que ofrece era necesario hacer:

```
1 NombreClase identificador = new NomClase();
```

Pero hay casos en los que, para inicializar correctamente la clase, hay que añadir un conjunto de valores entre los paréntesis, de forma idéntica a cómo se haría al invocar un método. Si no se ponen estos parámetros cuando proceda, habrá un error de compilación.

```
1 NombreClase identificador = new NomClase(parámetros);
```

De hecho, sin decirlo explícitamente ya ha trabajado anteriormente con un caso como éste, ya que es apenas lo que ocurre con la clase Scanner. Para inicializarla correctamente necesita un parámetro indicando qué sistema de entrada debe procesar.

```
1 Scanner lector = new Scanner(System.in);
```

En Java, el texto System.in
identifica la entrada
estándar, que por defecto
es el teclado.

Para ver si es necesario o no incluir un parámetro al inicializar una clase, será necesario buscarlo en su documentación. Concretamente, en el apartado llamado resumen de constructores (Constructor Summary) indica la sintaxis de la parte derecha de la inicialización. El nombre de este apartado se debe a que, formalmente, cuando se inicializa una clase se dice que se invoca a uno de sus constructores. Se puede elegir cualquiera de las opciones ofrecidas en la lista de constructores, pero según se elija, la clase puede comportarse de una manera diferente.

La figura 1.6 muestra este apartado para la clase Random. En este caso, se puede ver que hay dos formas diferentes de inicializar la clase antes de llamar a sus métodos. Se puede hacer como siempre, sin parámetros, Random(), o usando un valor de tipo long como parámetro, Random(long seed).

Figura 1. 6. Constructores de la clase Random

Constructor Summary	
Random ()	Creates a new random number generator.
Random (long seed)	Creates a new random number generator using a single long seed.

Reto 4. Modifique el programa anterior para mostrar por pantalla dos valores reales aleatorios de modo que la clase Random se inicie con un parámetro de tipo long. Este nuevo programa debería formar parte de un package llamado unidad5.apartado1.retos. Ejecutarlo varias veces. ¿Cuál es la diferencia entre usar un constructor u otro al inicializar la clase?

1.4 Métodos estáticos

Por norma general, para poder invocar métodos de una clase es necesario inicializarla previamente. Ahora bien, entre todos los métodos proporcionados dentro de las clases disponibles en la API de Java, existe un pequeño subconjunto con una peculiaridad.

Los métodos estáticos pueden ser invocados sin tener que inicializar su clase. Se identifican porque en la documentación aparecen marcados con la palabra clave `static`.

Nuevamente, esta sección se centra en el uso dentro del contexto de la API de Java, y no en cómo generarlos en sus clases o cuál es su justificación dentro de Java. Si bien la explicación es genérica, se centrará en las dos clases más importantes basadas en este tipo de método, que pueden ser de gran ayuda para hacer sus programas: `java.lang.Math` y `java.util.Arrays`. Si mira la documentación de estas dos clases, verá que casi todos sus métodos son estáticos. En la figura 1.7 puede ver un detalle de algunos de los métodos de la clase `Math`. Fíjese cómo están identificados con la palabra `static` al inicio de su declaración.

Figura 1. 7. Métodos estáticos de la clase `Math`

static long	<code>abs(long a)</code> Returns the absolute value of a long value.
static double	<code>acos(double a)</code> Returns the arc cosine of a value; the returned angle is in the range 0.0 through π .
static double	<code>asin(double a)</code> Returns the arc sine of a value; the returned angle is in the range $-\pi/2$ through $\pi/2$.
static double	<code>atan(double a)</code> Returns the arc tangent of a value; the returned angle is in the range $-\pi/2$ through $\pi/2$.

Para invocar estos métodos, la correcta sintaxis es poner como prefijo el nombre de la clase directamente, en lugar del identificador fruto de la inicialización. Aparte de esto, su comportamiento es idéntico a cualquier otro método (como una expresión que evalúa a un resultado).

```
1 NombreClase.nombreMetodo(parámetros);
```

Por ejemplo, dado el método estático `sqrt` definido en la clase `Math`, que acepta un único parámetro de entrada de tipo real, para invocarlo y guardar el resultado en una variable sólo sería necesario el código que se muestra a continuación. En ese caso, se calcula la raíz cuadrada de 36.

```
1 double resultado = Math.sqrt(36);
```

Fíjese cómo no es necesaria ninguna inicialización previa y, para invocarlos, se usa antes del identificador del método el nombre de la clase, y no de una variable.

1.4.1 La clase Math

Esta clase pertenece al package `java.lang`, y por tanto puede ser usada sin haber importarla. Ofrece un abanico de métodos estáticos para realizar operaciones matemáticas avanzadas, que no se pueden llevar a cabo con ninguno de los operadores aritméticos de que dispone Java (suma, resta, multiplicación, división y módulo).

A continuación se muestran algunos de los métodos más relevantes, asociados a operaciones matemáticas relativamente comunes y simples.

Redondeo

Un método interesante es el que permite redondear valores reales.

`round(valor)`, sólo tiene como parámetro de entrada un valor real (ya sea de tipo `float` o `double`) y evalúa siempre un entero (`int`). El redondeo se hace en el alza o bajista dependiendo de si su parte decimal es superior o igual a 0.5, o menor, respectivamente.

Por ejemplo, suponga que desea hacer un programa que, dado un valor real al azar entre 0 y 1, muestre su redondeo (al alta o a la baja, según el valor decimal es menor o superior a 0.5). El código de este programa sería lo que se muestra a continuación. En este ejemplo, fíjese en la diferencia en el código por lo que hace a la forma en que se llama en los métodos de las clases `Random` y `Math`.

```
1 package unidad5.apartado1.ejemplos;
2 importe java.util.Random;
3 public class ArrodonirReal {
4     public static void main(String[] args) {
5         ArrodonirReal programa = new ArrodonirReal();
6         programa.inicio();
7     }
8     public void inicio() {
9         //Inicialización de Random
10        Random rnd = new Random();
11        //Uso para generar un real entre 0 y 1
12        double valor = rnd.nextDouble();
13        System.out.println("El valor real generado es " + valor);
14        //Uso del método estático. No es necesario inicializar nada, se puede usar directamente.
15        //No es necesario importarla, ya que pertenece a java.lang
16        long redondeado = Math.round(valor);
17        System.out.println("El valor redondeado es " + redondeado);
18    }
19 }
```

Elevar a exponente y raíz cuadrada

Aunque otros lenguajes sí disponen de operador aritmético para elevar a un exponente (a^b), en Java no hay. Por eso, la clase `Math` ofrece el método estático para poder realizar esta operación.

`pow(base, exponente)` acepta dos parámetros, que sólo pueden ser de tipos `double`, de modo que su invocación evalúa el primer elevado al segundo (`baseexponente`).

Junto a este método, también se dispone de lo contrario, en este caso sólo por al cálculo de la raíz cuadrada.

La palabra `sqrt` abrevia `square root`, "raíz cuadrada" en inglés.

`sqrt(valor)` sólo tiene un parámetro, que sólo puede ser un real de tipo `double`, el cual evalúa el valor real resultante de hacer el cálculo de la raíz cuadrada de valor.

Hay que tener cuidado de no invocar nunca este método con un parámetro de entrada negativo, puesto que el resultado será siempre incorrecto.

Apenas se muestra un ejemplo de su uso, donde se aplica el teorema de Pitágoras por calcular la hipotenusa de un triángulo:

```
1 package unidad5.apartado1.ejemplos;
2 public class Pitagores {
3     public static void main (String[] args) {
4         Pitagores programa = new Pitagores();
5         programa.inicio();
6     }
7     public void inicio() {
8         double base = 4.5;
9         double altura = 10;
10        //Se eleva al cuadrado cada lado del triángulo
11        double sumaCuadrados = Math.pow(base,2) + Math.pow(altura, 2);
12        double hipotenusa = Math.sqrt(sumaCuadrados);
13        System.out.println("El valor de la hipotenusa es " + hipotenusa);
14    }
15 }
```

Máximo y mínimo

Para terminar este pequeño muestrario, es posible calcular rápidamente el máximo o el mínimo entre dos valores numéricos usando la clase `Math`, ya que también ofrece métodos para hacerlo.

`max(a, b)` usa dos parámetros de entrada de cualquier tipo primitivo numérico (`int`, `long`, `float` o `double`) y evalúa el valor máximo de entre ambos. Hay que tener en cuenta que los dos parámetros de entrada siempre deben ser del mismo tipo, y el tipo del valor resultante es el mismo que el de los dos parámetros de entrada. Si se comparan dos `long`, el resultado es también un `long`, por ejemplo.

`min(a, b)` es el mismo que el anterior, pero para el cálculo del mínimo.

A continuación se muestra un pequeño programa que muestra cómo invocarlos.

```
1 package unidad5.apartado1.ejemplos;
2 public class MinMax {
3     public static void main (String[] args) {
4         MinMax programa = new MinMax();
5         programa.inicio();
6     }
7     public void inicio() {
8         double resReal = Math.max(1.2, 4.5);
9         System.out.println("El valor máximo es " + resEnter = + resReal);
10        Math.min(-8, 20);
11        System.out.println("El valor mínimo es " + resEnter);
12    }
13 }
```

```
12 }  
13 }
```

1.4.2 La clase Arrays

Esta clase pertenece al package `java.util` (como `Random` y `Scanner`). Ofrece métodos estáticos para realizar operaciones típicas con arrays (ordenaciones, copias, búsquedas, llenar rangos de posiciones con ciertos valores, etc.). Esto la hace muy interesante, ya que ahorra mucho trabajo y permite generar código de forma sencilla. Merece la pena conocer a grandes rasgos qué métodos contiene y saber invocarlos correctamente. Ahora bien, para usarlos es necesario tener presente que algunos de ellos manipulan los parámetros de entrada de tipo compuesto, en este caso arrays. Por tanto, al finalizar la invocación, el array original habrá cambiado.

A continuación se listan algunas de las tareas que se pueden conseguir usando métodos estáticos de esta clase. Ciertamente, una vez los domine, verá que esto hace el código de los programas mucho más sencillo que cuando debe codificar estas tareas usted mismo!

1. Ordenación

Poder ordenar los datos en un array puede ser muy cómodo para mostrarlos por pantalla y para que los interprete. Por tanto, es una operación común. `Arrays` ofrece un método para realizar esta operación fácilmente.

`sort(array)` sólo tiene un parámetro de entrada, que puede ser un array de cualquiera tipos de datos primitivo o de cadenas de texto. Al finalizar la invocación, sus elementos estarán ordenados (por valor numérico o alfabéticamente, según el tipo de datos). Este método no evalúa nada (retorna `void`), ya que trabaja directamente sobre el parámetro de entrada, modificando su contenido.

El ejemplo siguiente muestra cómo ordenar muy fácilmente un array de cadenas de texto. Fíjese que este método no evalúa ningún resultado, su llamada simplemente transforma el contenido del parámetro de entrada.

La existencia de la clase `Arrays` no hace menos importante dominar los esquemas de manipulación de arrays.

```
1 package unidad5.apartado1.ejemplos;  
2 importe java.util.Arrays;  
3 public class OrdenaArrayText {  
4     public static void main (String[] args) {  
5         OrdenaArrayText programa = new OrdenaArrayText();  
6         programa.inicio();  
7     }  
8     public void inicio() {  
9         String[] array = {"Uno", "Dos", "Tres", "Cuatro", "Cinco"};  
10        Arrays.sort(array);  
11        System.out.println("Los elementos ordenados son:");  
12        for (int i = 0; i < array.length; i++) {  
13            System.out.println(array[i]);  
14        }  
15    }  
16 }
```

2. Búsqueda

Uno de los esquemas más típicos de operación con arrays es, de hecho, buscar dónde hay una posición con un valor concreto, por lo que también es útil disponer de un método que lo permita.

`binarySearch(array, clave)` trabaja a partir de dos parámetros de entrada: uno array de cualquier tipo primitivo o `String`, que debe haber sido previamente ordenado, y el valor a buscar, que es la clave de la búsqueda. Si hay un elemento con el valor buscado, el método evalúa el número de la posición donde se sitúa la primera aparición de la clave (un entero), o un valor negativo si no se ha encontrado ninguna. El valor negativo permite averiguar la posición de inserción, dentro del array, del valor no encontrado, por tal que el array siga ordenado. Para averiguar la posición de inserción es necesario calcular $-(\text{valor negativo}) - 1$.

Para poder invocarlo correctamente, las posiciones del array y el valor a buscar deben ser del mismo tipo exactamente. También es indispensable que el array tenga los sus elementos ordenados, ya que de lo contrario no se puede garantizar la corrección de la búsqueda. Por último, siempre debe tener cuidado de comprobar si realmente se ha encontrado la clave y, por tanto, si el método ha evaluado un valor positivo o negativo, ya que en el segundo caso se trata de una posición inválida en el array. En el siguiente ejemplo se puede ver esto:

```
1 package unidad5.apartado1.ejemplos;
2 importe java.util.Arrays;
3 public class BuscarArray {
4     public static void main(String[] args) {
5         BuscarArray programa = new BuscarArray();
6         programa.inicio();
7     }
8     public void inicio() {
9         int[] array = {2, 5, 8, 1, 4, 2, 5, 3, 10};
10        //Primero hay que ordenar
11        Arrays.sort(array);
12        int pos = Arrays.binarySearch(array, 5);
13        System.out.println("Hay un 5 en la posición " pos =          + pos);
14        Arrays.binarySearch(array, 6);
15        System.out.println("Hay un 6 en la posición " + pos);
16    }
17 }
```

3. Copia

Si hace memoria, recordará que una de las propiedades de los tipos compuestos en Java es que no se pueden realizar copias simplemente con una asignación. Lo que se obtiene son dos variables con nombres distintos que permiten acceder a los mismos datos. Si se quiere una copia de un array, es necesario declarar uno nuevo y copiar todos los valores del original, posición por posición. Otra opción es usar el método que ofrece la clase `Arrays`.

`copyOfRange(array, posInicio, posFi)` permite crear copias de un conjunto de posiciones de un array (todas, o sólo una parte). Este array puede contener cualquier tipo primitivo o cadenas de texto. Requiere tres parámetros, el array con las posiciones a copiar, la posición origen y la posición destino. Al invocarlo, éste evalúa un nuevo array con $(\text{posFi} - \text{posInici})$ posiciones, en cada una

de las cuales están copiados los valores entre estas posiciones (desde posInicio hasta posFi - 1).

Hay que tener cuidado con que las posiciones origen y destino sean válidas (positivas y dentro del rango de posiciones). También debe cumplirse que la posición destino sea mayor que el origen. De lo contrario, se producirá un error en la ejecución del programa. Por último, recuerde que para almacenar el resultado será necesario asignarlo a una variable de tipo array.

La mejor forma de verlo es con un ejemplo, en el que el resultado de hacer la operación de copia queda almacenada en la variable copia.

```
1 package unidad5.apartado1.ejemplos;
2 importe java.util.Arrays;
3 public class CopiaArray {
4     public static void main(String[] args) {
5         CopiaArray programa = new CopiaArray();
6         programa.inicio();
7     }
8     public void inicio() {
9         int[] origen = {2, 5, 8, 1, 4, 2, 5, 3, 10, 8};
10        //Si se desea copiar de la posición 1 a la 5
11        int[] copia = Arrays.copyOfRange(origen, 1, 6);
12        System.out.println("Los elementos de la copia son: ");
13        for (int i = 0; i < copia.length; i++) {
14            System.out.println(copia[i]);
15        }
16    }
17 }
```

4. Igualdad

Comparar dos arrays implica recorrerlos e ir comprobando posición por posición si todas son iguales. También hay un método que ya permite realizar esta comprobación fácilmente.

ecual (array1, array2) tiene dos parámetros de entrada, que son dos arrays de cualquier tipo de datos primitivo o cadena de texto. Su invocación evalúa un booleano que es true si ambos son iguales y false si no es el caso.

Los valores almacenados en ambos arrays deben ser del mismo tipo de datos (no se puede comparar un array de enteros y uno real, por ejemplo). El ejemplo siguiente muestra el funcionamiento de este método.

```
1 package unidad5.apartado1.ejemplos;
2 importe java.util.Arrays;
3 public class ComparaArray {
4     public static void main(String[] args) {
5         ComparaArray programa = new ComparaArray();
6         programa.inicio();
7     }
8     public void inicio() {
9         double[] origen = {4.5, 1.2, 3.0, 1.4, 3.5};
10        //Se hace una copia exacta
11        double[] copia = Arrays.copyOfRange(origen, 0, origen.length);
12        boolean iguales = Arrays.equals(origen, copia);
13        System.out.println("¿Son iguales? " + iguales);
14        //Se modifica la copia
15        copia[3] = 4.6;
16        iguales = Arrays.equals(origen, copia);
17        System.out.println("Y ahora, ¿son iguales? " + iguales);
18    }
19 }
```

```
18     }  
19 }
```

5. Transformación a texto

Por último, una tarea muy típica es mostrar el contenido de un array por pantalla (como se ve ya en los ejemplos recientes). Por eso, es útil transformarlo a una cadena de texto.

`toString(array)`, dado un array cualquiera como parámetro de entrada, evalúa una cadena de texto en la que se enumeran todos sus valores. El formato de la cadena resultante es una lista de valores separados por comas, entre claves: "[valor1, valor2, ..., valorN]".

Vea un ejemplo a continuación.

```
1 package unidad5.apartado1.ejemplos;  
2 importe java.util.Arrays;  
3 public class MostraArray {  
4     public static void main(String[] args) {  
5         MostraArray programa = new MostraArray();  
6         programa.inicio();  
7     }  
8     public void inicio() {  
9         int[] array = {2, 5, 8, 1, 4, 2, 5, 3, 10, 8};  
10        String text = Arrays.toString(array);  
11        System.out.println("El array contiene los valores: " + text);  
12    }  
13 }
```

Reto 5. Cree una clase que pertenezca al package `unidad5.apartado1.retos`, el que haga lo siguiente. Se genera un array de 10 posiciones y cada una de ellas se inicializa con un valor al azar entre 1 y 10, y se muestran por pantalla. Entonces, por cada valor escrito entre las posiciones 0 a 4, se busca si éste es en alguna de las posiciones entre la 5 y la 9. Cada vez que encuentra uno de los valores, lo anuncia por pantalla. Utilice todos los métodos de la clase `Arrays` que sea posible.

1.5 Documentación de programas en Java

Cuando el tamaño de los programas crece y éstos se hacen complicados, compuestos por conjuntos de clases dentro de varios packages, y cada una con sus métodos, es importante disponer de alguna forma de poder saber fácilmente para qué sirve cada cosa. Algún mecanismo que haga fácil evaluar si se pueden reutilizar en otros programas, o reanudar el hilo de un programa ya hecho con anterioridad si más adelante desea hacer enmiendas a su código fuente.

Precisamente, un aspecto bastante importante, pero a menudo desatendido, durante el desarrollo de un programa es documentar correctamente su funcionamiento. Es necesario mantener un documento en el que se explique cómo funcionan todos los módulos y de qué se componen (en el caso de Java: packages, clases y métodos), que debe de estar actualizado siempre en la última versión del programa. Siempre debe tener

presente que, si bien mientras él está desarrollando le parecerán evidentes todos los aspectos relativos a su funcionamiento, pasado un tiempo se olvidará. Entonces, si más adelante debe volver a hacer alguna enmienda o alguna modificación, puede resultar más trabajo volver a entender qué hacía cada bloque de código que la tarea propiamente encomendada.

Siempre debe tener bien presente, como buen hábito de programación, que puede darse el caso de que, de hecho, no sea usted mismo quien tiene que entender sus programas, sino terceras personas. El abanico de posibilidades bajo las cuales se puede dar esta circunstancia puede ir desde un compañero de trabajo que debe enmendar un pedazo de código que hicisteis hasta un profesor que está evaluando su ejercicio y necesita que sea fácil de entender y saber qué hace cada parte. Simplemente, póngase en el lugar de tener que entender cientos de líneas de código fuente que no ha hecho usted para hacerle una idea de para qué sirven. En programas complejos, no es una tarea sencilla.

Aunque no es tan evidente como generar código fuente, facilitar la labor de comprensión del código también es una parte integral del proceso de desarrollo de un programa.

Un ejemplo muy claro de la utilidad de disponer de una buena documentación es la de la API de Java, en la que se enumeran todas las clases, bibliotecas y métodos disponibles, explicando con detalle, para cada caso, para qué sirven sus parámetros de entrada y el valor de retorno.

1.5.1 Javadoc

Existen muchas aproximaciones para llevar a cabo un proceso de documentación cuidadosa de un programa. De hecho, en muchas ocasiones puede llegar a ser más complicado mantener la documentación actualizada que generar el código fuente. Esto se debe a que se trata de generar un documento adicional que es necesario modificar cada vez que hay cambios en el código. Esto hace que el proceso sea farragoso.

Java intenta paliar este hecho mediante un mecanismo que permite generar documentación automáticamente a partir del texto escrito en el código fuente, por lo que sólo es necesario gestionar un único archivo para tener la información actualizada.

El javadoc es una herramienta auxiliar proporcionada por el kit de desarrollo de Java (JDK), que permite generar automáticamente documentación relativa a clases Java a partir de comentarios dentro de su código fuente.

La herramienta javadoc procesa los archivos de código fuente de las clases de un programa, por lo que busca ciertas palabras clave distintivas insertadas en los comentarios ya partir de éstas generar automáticamente documentación en formato HTML. Esto puede ahorrarle mucho trabajo si mientras vaya haciendo el código fuente del programa sueldo

Los IDE suelen disponer de una opción para integrar en su entorno la ejecución de la herramienta javadoc.

cuidadosos y ya va creando estos comentarios, usando las palabras clave correspondientes. Codificación y documentación se convierten en tareas paralelas.

De hecho, la documentación de la API de Java estuvo generada en su momento usando esta herramienta. Todas las clases que vienen con las bibliotecas de Java se programaron comentando su código fuente de acuerdo al formato adecuado. Al terminar, sólo fue necesario ejecutar javadoc para crear el HTML con toda la documentación.

1.5.2 Sintaxis general

Todo el texto con la información requerida para generar documentación usando javadoc se escribe dentro de comentarios en el código fuente de acuerdo al formato empleado para escribir múltiples líneas, que es lo que se muestra a continuación. No se pueden usar dobles contrabarras (que sirven para comentar una línea individual).

```
1 /** Línea 1 de comentario.  
2  * Línea 2 de comentario.  
3  * ...  
4  * Línea N del comentario.  
5 **/
```

Estos comentarios se pueden insertar en las líneas inmediatamente antes tanto de la declaración de la clase (public class...) como de los métodos. Lo que se escriba en las diferentes líneas del comentario será la explicación detallada de esa clase o método en la documentación final.

Por ejemplo, el siguiente esqueleto de clase aportaría la documentación de la clase propia y de sus dos métodos.

```
1 package unidad5.apartado2.ejemplos;  
2 /** Apenas este texto será la documentación de la clase.  
3  * Lo que esté escrito se incluirá directamente en el HTML resultante.  
4  * Esta es una clase bien documentada.  
5 **/  
6 public class ClaseComentada {  
7     /** Éste es su primer método.  
8     * Sólo se muestra el esqueleto, no es necesario código de momento.  
9     * Es importante ver cómo funcionan los comentarios para poder usar el javadoc  
10    * ...  
11    */  
12    public void unMetode() {  
13        ...  
14    }  
15    /** Otro método, más escaso en documentación.  
16    */  
17    public void unOtroMetodo() {  
18        ...  
19    }  
20 }
```

Todo el texto escrito en los comentarios se transforma en texto normal en HTML, en un mismo párrafo. Si se quieren añadir efectos especiales (negritas, cursiva. etc.) o estructuras de texto algo más complejas (como listas ordenadas o enumeradas), en el texto de estos comentarios es posible usar pedidos en HTML.

Por ejemplo, si se sabe que el mandato HTML `<b>...</b>` sirve para hacer que uno texto aparezca en negrita, y que el comando `<ol>...</ol>` permite crear una lista enumerada, se podría hacer:

```
1 /** Este es un método extra.
2  * Ahora contiene una <b>lista de elementos</b>.
3  * <ol>
4  * <li> Ítem 1.
5  * <li> Ítem 2.
6  * <li> Ítem 3.
7  * <li> Ítem 4.
8  * </ol>
9  */
10 public void unTercerMetode() {
11     ...
12 }
```

Al usar órdenes es necesario tener cuidado, ya que debe tener presente que el texto que ha escrito se acabará incorporando a un documento HTML general. Por tanto, es mejor limitarse a las órdenes más simples, y sobre todo, nunca usar órdenes vinculadas a encabezamientos (`<h1>...</h1>`, etc.).

Reto 6. Intente ejecutar el javadoc desde su entorno de desarrollo para generar la documentación del ejemplo. Tenga en cuenta que para que funcione no es necesario que haya instrucciones dentro de los métodos, los puede dejar vacíos. Sólo hace falta que haya comentarios siguiendo este formato.

1.5.3 Palabras clave

Hay algunos aspectos de una clase o un método que son especialmente relevantes, como su autor, parámetros de entrada o valor de retorno. Aparte de una descripción general, vale la pena disponer de una lista clara de cuáles son y el valor esperado, de modo que destaquen más en el documento HTML resultante. Por eso, hay un seguido de palabras clave que se pueden usar una vez finalizado el texto, por lo que se indica en el javadoc esta información adicional.

Todas las palabras clave quedan identificadas porque se preceden de un símbolo "@", de modo que no puedan ser confundidas por texto normal, parte de la descripción. Después de la palabra clave, se puede escribir una línea de texto cualquiera que se considera su información asociada.

A continuación verá algunas de las palabras clave más usadas.

- **@author**. Esta palabra clave sólo se usa después del texto asociado a la descripción de la clase. Indica quién ha sido su autor. Por ejemplo:

```
1 package unidad5.apartado2.ejemplos;
2 /** Apenas este texto será la documentación de la clase.
3  * Lo que esté escrito se incluirá directamente en el HTML resultante.
4  * Esta es una clase bien documentada
5  * @author IOC
6  */
```

```

7 public class ClaseComentada {
8 ...
9 }

```

- **@param.** Esta palabra clave sólo se utiliza después del texto asociado en la descripción de un método y describe uno de sus parámetros. Será necesario añadir tantas palabras clave de este tipo, en diferentes líneas consecutivas, cómo parámetros tiene el método. El texto debe empezar por el identificador del parámetro de entrada que se desea describir, ya continuación se escribe su descripción, dónde habría que explicar para qué sirve.

```

1 /** Un método con parámetros.
2  * @param unParametro Este parámetro sirve para hacer algo.
3  * @param otro Parametro Este parámetro diferente sirve para hacer una
   otra cosa.
4  */
5 public void unMetodeParametrizado(int unParametre, int otroParametre) {
6  ...
7  }

```

- **@returns.** Esta palabra clave sólo se usa después del texto asociado a la descripción de un método y describe qué evalúa el método (el valor devuelto). Normalmente se escribe después de las palabras clave **@params**, si las hay. Si un método no devuelve nada (void), no es necesario incluirlo. El texto asociado es directamente la descripción.

```

1 /** Un método con parámetros.
2  * @param unParametro Este parámetro sirve para hacer algo.
3  * @param otro Parametro Este parámetro diferente sirve para hacer una
   otra cosa.
4  * @returns Un valor que depende de los dos parámetros.
5  */
6 public int unMetodeParametrizado(int unParametro, int otroParametro) {
7  ...
8  }

```

- **@see.** Esta palabra clave se puede utilizar tanto después de la descripción de un método como de una clase. Sirve para referirse a la página de documentación de otras clases dentro del mismo programa, en los casos de programas modulares compuestos por distintas clases. Básicamente dice: "Por además información, ver también la documentación de..." No sirve para referirse a cualquier clase, sólo funciona con clases de las que se dispone el código fuente comentado en este formato. Dentro de un IDE, normalmente serán clases dentro del mismo proyecto.

Como texto asociado, se debe poner el nombre calificado de la clase a que se refiere.

```

1 /** Apenas este texto será la documentación de la clase.
2  * Lo que esté escrito se incluirá directamente en el HTML resultante.
3  * Esta es una clase bien documentada.
4  * @author IOC
5  * @see utilidades.arrays.CalculsArrays
6  */
7 public class ClaseComentada {
8 ...
9 }

```

1.6 Solución de los retos propuestos

Reto 1

```
1 public class MediaMaxima {
2     public static void main(String[] args) {
3         MediaMaxima programa = new MediaMaxima();
4         programa.inicio();
5     }
6     public void inicio() {
7         double[] arrayA = {2, 10.5, 6.3, 4.9, 0, 7.1, 8.3};
8         double[] arrayB = {9.4, 6.0, 2.2, 1.0, 8.1};
9         //Para llamar a los métodos es necesario inicializar la clase que los contiene
10        CalculsArrayReals calculador = new CalculsArrayReals();
11        //Una vez hecho, hay que llamarlos usando como prefijo el identificador
12        double mediaA = calculador.calcularMedia(arrayA);
13        double mediaB = calculador.calcularMedia(arrayB);
14        if (mediaA > mediaB) {
15            System.out.println("arrayA tiene una media más alta.");
16        } else if (mediaA < mediaB) {
17            System.out.println("arrayB tiene una media más alta.");
18        } else {
19            System.out.println("¡Las dos medias son iguales!");
20        }
21    }
22 }
```

Reto 2

```
1 //----- Archivo RegistroNotes.java -----
2 package unidad5.apartado1.retos;
3 public class RegistroNotes {
4     public static void main(String[] args) {
5         RegistroNotes programa = new RegistreNotes();
6         programa.inicio();
7     }
8     public void inicio() {
9         double[] notas = {2.0, 5.5, 7.25, 3.0, 9.5, 8.25, 7.0, 7.5};
10        //Para llamar a los métodos es necesario inicializar la clase que los contiene
11        CalculsArrayReals calculador = new CalculsArrayReals();
12        //Una vez hecho, hay que llamarlos usando como prefijo el identificador
13        double max = calculador.calcularMaximo(notas);
14        double min = calculador.calcularMinimo(notas);
15        double media = calculador.calcularMedia(notas);
16        System.out.println("La nota máxima es " + max + ".");
17        System.out.println("La nota mínima es " + min + ".");
18        System.out.println("La media de las notas es " + media + ".");
19    }
20 }
```

```
1 //----- Archivo CalculsArrayReals.java -----
2 package unidad5.apartado1.retos;
3 public class CalculsArrayReals {
4     public double calcularMaximo(double[] array) {
5         double max = array[0];
6         for (int i = 1; i < array.length; i++) {
7             if (max < array[i]) {
8                 max = array[i];
9             }
10        }
11        return max;
12    }
13    public double calcularMinim(double[] array) {
14        double min = array[0];
```

```
15     for (int i = 1; y < array.length; i++) {
16         if (min > array[i]) {
17             min = array[i];
18         }
19     }
20     return min;
21 }
22 public double calcularMedia(double[] array) {
23     double suma = 0;
24     for (int i = 0; y < array.length; i++) {
25         suma = suma + array[i];
26     }
27     return suma/array.length;
28 }
29 }
```

Reto 3

```
1 //----- Archivo RegistroNotes.java -----
2 package unidad5.apartado1.retos;
3 importe utilidades.arrays.CalculsArraysReals;
4 public class RegistroNotes {
5     //El código no cambia
6     ...
7 }
```

```
1 //----- Archivo CalculsArrayReals.java -----
2 package utilidades.arrays;
3 public class CalculsArrayReals {
4     //El código no cambia
5     ...
6 }
```

Reto 4

```
1 package unidad5.apartado1.retos;
2 importe java.util.Random;
3 public class RealsAleatorios {
4     public static void main(String[] args) {
5         RealsAleatorios programa = new RealsAleatorios();
6         programa.inicio();
7     }
8     public void inicio() {
9         //Iniciación. Por ejemplo, se utiliza el valor 100 como "raíz" (seed).
10        Random rnd = new Random(100L);
11        //Uso
12        double valorA = rnd.nextDouble() * 100;
13        double valorB = rnd.nextDouble() * 100;
14        System.out.println("Se han generado los valores" + valorA + " y " + valor B);
15    }
16 }
```

Reto 5

```
1 package unidad5.apartado1.retos;
2 importe java.util.Arrays;
3 importe java.util.Random;
4 public class BuscarValores {
5     public static void main(String[] args) {
6         CercaValors programa = new CercaValors();
7         programa.inicio();
8     }
9     public void inicio() {
10        int[] array = new int[10];
11        Random rnd = new Random();
```

```
12 //Rellenamos el array
13 for (int i = 0; i < array.length; i++) {
14     array[i] = rnd.nextInt(11);
15 }
16 //Se divide el array en dos
17 int[] arrayValors = Arrays.copyOfRange(array, 0, 5);
18 int[] arrayBuscar = Arrays.copyOfRange(array, 5, array.length);
19 Arrays.sort(arrayBuscar);
20 System.out.print("Valores a buscar: ");
21 System.out.println(Arrays.toString(arrayValors));
22 System.out.print("Array donde se busca: ");
23 System.out.println(Arrays.toString(arrayBuscar));
24 //Se realizan las búsquedas
25 for (int i = 0; i < arrayCercar.length; i++) {
26     int valor = arrayValors[i];
27     int pos = Arrays.binarySearch(arrayBuscar, valor);
28     if (pos >= 0) {
29         System.out.println("En la posición" + pos + " está el valor " + valor);
30     }
31 }
32 }
33 }
```

Reto 6

Los pasos para realizar la tarea dependen del entorno de desarrollo usado.

2. Creación de una aplicación modular. El juego de combates en la arena

Durante el proceso de aprender a programar se van presentando diferentes conceptos que se consideran importantes, desde aspectos más teóricos o metodológicos (qué es un tipo de datos, programación estructurada, diseño descendente, etc.), hasta aquellos muy ligados a la sintaxis del lenguaje (inicialización de variables, invocación de métodos, sentencias, etc.). Cuando se escriben programas complejos, es imprescindible saber combinar todas estas piezas para llegar a buen puerto. Desgraciadamente, a menudo se puede encontrar que resulta factible entender cada concepto de manera individual, y resolver problemas simples, pero no es sencillo ver cómo encajar todas las piezas en problemas más complejos.

En esta unidad se plantea resolver un problema con un alto grado de complejidad al explicar el funcionamiento del juego de combates en la arena. El objetivo es que le sirva de ejemplo aplicado, y mostrar cómo todas estas pequeñas piezas que han ido aprendiendo dentro del campo de la programación se pueden combinar para poder generar un programa de cierta envergadura, así como aprender algunos criterios que se consideran acertados a la hora de tomar algunas decisiones. Para crear este programa, se hará especial énfasis en el concepto de modularidad, de forma que su código fuente se componga de varias clases y pueda verse claramente su justificación.

2.1 El juego de combates en la arena

El programa que sirve como hilo argumental de este apartado es un juego, en el que el jugador se va enfrentando a varios adversarios en una arena. Cada combate se divide en rondas, al inicio de las cuales el jugador y su adversario eligen secretamente una estrategia a seguir. Cada ronda puede seguir una estrategia distinta. Según las estrategias elegidas por cada uno, el combate se irá resolviendo más favorablemente hacia uno u otro, hasta que finalmente se considere que uno de los dos ha sido derrotado. Si se derrota al adversario, se otorga una puntuación al jugador. Si el jugador es derrotado, termina la partida. El objetivo final del jugador es sobrevivir diez combates, logrando la máxima puntuación posible en el proceso.

Tanto para mostrar datos al usuario como para introducir las órdenes del jugador, se usa sólo texto.

2.1.1 Descripción detallada del programa

Al tratarse de un problema más complejo, vale la pena dedicar un espacio a describir detalladamente el problema y ver en qué consiste exactamente el juego, en el que aprovechará la capacidad de un ordenador para resolver rápidamente el tratamiento de unos datos para implementar un sistema de combate entre dos luchadores un poco elaborado, inspirado en los sistemas empleados en los CRPG (Computer Role Play) ni mucho menos en el nivel de los juegos modernos.

Puede encontrar un poco más de información sobre qué es un CRPG en la dirección <http://goo.gl/cxZpb>.

Atributos de los luchadores

Para describir a todos los luchadores, tanto el jugador como sus adversarios, éstos disponen de una serie de atributos que indican su estado en todo momento. Algunos de estos atributos sirven para establecer cómo progresa el combate y pueden ver modificados sus valores. A continuación se enumeran:

- Nombre: el nombre del luchador. Para el jugador es "Aventurero", mientras que para los adversarios se referirá a criaturas fantásticas ("Nan", "Ogre", "Hidra", etc.)
- Nivel: indicador general de la capacidad de combate del luchador.
- Puntos: los puntos que ha acumulado el luchador hasta el momento.
- Puntos de Vida (PV): la energía del luchador actual, que puede variar a lo largo del combate. Cuando llega a 0 o menos, se considera derrotado.
- Puntos de Vida Máximos: valor máximo que pueden tener los puntos de vida en cualquier momento.
- Ataque: su capacidad de llevar a cabo con éxito estrategias ofensivas. Se usa para resolver el resultado de una ronda de combate.
- Defensa: al igual que la anterior, pero para estrategias defensivas.

Combate entre luchadores

El programa se basa en que el jugador va realizando un combate tras otro contra distintos adversarios. Para ganar, debe sobrevivir a diez combates. Hasta que no termina un combate, y se ha decidido si el jugador le ha ganado o lo ha perdido, no se empieza uno nuevo.

Al inicio de cada combate, se muestra el estado actual del jugador, el valor actual de todos sus atributos, preguntándole contra qué adversario quiere luchar. El jugador debe escribir el nombre de un adversario. Si ese nombre no se encuentra entre el de los adversarios disponibles en el juego, iniciará un combate contra un elegido al azar entre todos los adversarios disponibles de su mismo nivel o uno de diferencia. Esto evita que,



Un combate a muerte entre dos luchadores

por sorpresa, se encuentre que debe luchar contra un adversario demasiado poderoso para él, imposible de ganar.

Si el nombre pertenece a algún adversario disponible, entonces se enfrenta contra él. En este caso, no existe ninguna restricción de nivel. El jugador puede elegir luchar contra adversarios mucho más o menos poderosos que él.

Este planteamiento está dispuesto de forma que, de entrada, un nuevo jugador no sabe el nombre de ningún adversario, ya que no se proporciona ninguna lista (a menos que haya hecho el programa o visto el código fuente, por supuesto). La intención es que vaya descubriendo nuevos nombres de adversarios a medida que va jugando partidas o hablando con amigos que también jueguen al juego.

Resolución de una ronda de combate

Cada combate se divide en una serie de rondas, en cada una de las cuales el jugador debe elegir qué estrategia desea usar. Al principio de cada ronda se muestra el estado actual tanto del jugador como de su adversario, de forma que sea posible evaluar qué vía de acción puede convenirle más llevar a cabo. Entonces, el jugador elige la estrategia entre cuatro posibles: Atacar, Defender, Engaño y Maniobra. Una vez elegida, el adversario elegirá la suya y se decidirá el resultado de la ronda.

Ante todo, hay que ver para cada luchador el grado de éxito de su estrategia. Si ha elegido Atacar o Engaño, representa que tira tantas monedas como su valor de Ataque. Si ha elegido Defender o Maniobra, hace lo mismo usando su valor de Defensa.

El grado de éxito será el número de caras obtenidas.

Al resolver la ronda, cada luchador puede recibir uno de los siguientes efectos. La gravedad de cada uno de ellos depende del grado de éxito del propio luchador o de su contrincante.

- Nada: no pasa nada.
- Dañado: el luchador pierde una cantidad de puntos de vida igual al grado de éxito del contrincante.
- Curado: el luchador recupera tantos puntos de vida, sin superar nunca el valor máximo, como su propio grado de éxito.
- Penalizado: el luchador ve penalizado su valor de ataque o defensa (se elige al azar) en tantos puntos como el grado de éxito del contrincante. La penalización nunca puede bajar el valor por debajo de 1. Este efecto dura hasta la próxima ronda múltiple de cinco (5, 10, 15, etc.). Entonces, devuelve a su valor original.

El efecto que recibe cada luchador depende de las interacciones entre las estrategias, de forma similar al juego de piedra, papel, tijeras. Dependiendo de la estrategia elegida y la del adversario, el resultado será distinto. La tabla 2.1 muestra el resultado de las interacciones entre estrategias. Para abreviar, "Jug" se refiere al jugador y "Adv" al adversario. Un indicador de "x2" significa que a la hora de resolver este efecto, se doblan los logros alcanzados por el contrincante.

El sistema del juego de combates en la arena está inspirado en el juego de rol de sobremesa "The Mouse Guard RPG" (por David Petersen).

Tabla 2. 1. Resolución de estrategias

Jug\Adv	Ataque	Defensa	Engaño	Maniobra
Ataque	Jugo y Adv: Dañado	Adv: Curado	Adv: Dañado	Adv: Dañado
Defensa	Juego: Curado	Jugo y Adv: Curado	Jugo: Dañado x2	Juego: Penalizado
Engaño	Juego: Dañado	Adv: Dañado x2	Jugo y Adv: Dañado	Juego: Penalizado
Maniobra	Juego: Dañado	Adv: Penalizado	Adv: Penalizado	Jugo y Adv: Penalizado

Ejemplo de ronda de combate

Por ejemplo, suponga que el jugador tiene ahora mismo 10 puntos de vida y sus valores de ataque y defensa son 4 y 3, respectivamente. El adversario tiene 6 puntos de vida y sus valores de ataque y defensa son 3 y 5 respectivamente. Ante todo, cada uno elige su estrategia. El jugador elige Ataque mientras que el adversario elige Maniobra. Esto significa que, para ver el grado de éxito, el jugador lanzará tantas monedas como su Ataque y el adversario, al haber elegido Maniobra, tantas como su defensa. El jugador tira 4 monedas y suponga que saca dos caras. El adversario tira 5 y suponga que obtiene cuatro.

Ahora es necesario ver el efecto de las estrategias. De acuerdo a la mesa, si el jugador elige Ataque y el adversario elige Maniobra, el resultado es que el adversario recibe el efecto de "Dañado" (Adv: Dañado). Se le descuentan tantos puntos de vida como el grado de éxito del jugador (2). Por tanto, ahora le quedan $6 - 2 = 4$ puntos de vida y termina esta ronda.

Se inicia una nueva ronda donde se muestra el estado de los dos luchadores y se elige una nueva...

Evidentemente, a la hora de elegir la estrategia, el ordenador no debería hacer trampas (ya que conocerá la del jugador antes de elegir la suya). Se puede elegir al azar, o siguiendo alguna táctica según su estado (defender más a menudo si le quedan pocos puntos de vida, engañar si el jugador defiende muy a menudo, etc.). Esto depende del grado de inteligencia que se desea que tenga el ordenador.

Resolución de la finalización del combate

El combate finaliza cuando, al terminar una ronda, uno de los luchadores tiene 0 o menos puntos de vida. Si se trata del jugador, se considerará derrotado. La partida termina y se muestra su puntaje final. Esta circunstancia incluye también el caso de empate (ambos luchadores han llegado a 0 puntos de vida). Si, de lo contrario, es el adversario el derrotado, al personaje se le otorga cierta cantidad de puntos, que se suman a los que ya dispone. Los puntos otorgados dependerán de los puntos del adversario y, normalmente, adversarios más difíciles tendrán siempre puntos.

Al otorgar puntos al jugador, si éste llega o supera un valor asociado a una centena (100, 200, 300, etc.), se considera que "sube de nivel" y se hace más poderoso. Cuando esto sucede, el jugador ve incrementado en un punto su nivel, su máximo de puntos de vida se incrementa en dos, y su ataque o defensa, uno de los dos elegido al azar, se incrementa en un punto. El jugador también es inmediatamente curado, recuperando todos los puntos de vida actuales hasta ese nuevo máximo.

Una vez otorgados los puntos y un posible incremento de su nivel, todas las penalizaciones actuales sobre el jugador desaparecen. Ahora bien, a menos que haya subido de nivel, éste no recupera ningún punto de vida. Comenzará el siguiente combate con exactamente los mismos puntos con los que ha finalizado éste.

Si éste era el décimo combate, la partida termina con un mensaje de felicitación y se muestra la puntuación final. De lo contrario, se vuelve a iniciar un nuevo combate.

2.1.2 Identificación de los datos a tratar

De la descripción del problema, los datos principales a tratar son los de los dos luchadores, el jugador y sus adversarios, que serán las mismas. Afortunadamente, la descripción del problema ofrece una visión clara de qué tipo de valores es necesario manipular (Nombre, Nivel, Vida, etc.). En este caso, dado que son bastantes valores, todos vinculados entre ellos, lo más factible es usar un array, de modo que se gestione un para el jugador y uno para el adversario. De esta forma, con un par de variables es sencillo disponer de todos los datos vinculados a ambos. Cada posición del array puede representar cada uno de los atributos.

Con el fin de elegir el tipo de datos a guardar, prácticamente todos los atributos de los luchadores son enteros, por lo que se pueden usar arrays de enteros para representarlos.

El único caso especial que rompe esta solución simple es el nombre, que sería una cadena de texto. Para resolver esto se puede usar la siguiente estrategia. Dentro del array se utiliza un entero para identificar el nombre, de modo que cada luchador siempre tenga un identificador distinto. Entonces, aparte, se puede usar una tabla de traducción de identificadores en cadenas de texto. El identificador de tipo entero de cada luchador dice cuál es el índice dentro de esta tabla de traducción donde se encuentra su nombre.

Si a simple vista no os queda claro cómo llevar a cabo esto, no se preocupe por ahora. Cuando llegue el momento de mostrar el código fuente para hacer esto, ya se verá mejor.

La tabla 2.2 muestra un resumen de una primera propuesta de estructura del array que representa a un luchador, según las posiciones de éste. Ahora bien, siempre debe ser conscientes de que, a medida que se va codificando el programa, se puede dar el caso de que se da cuenta de que es necesario modificarla.

Tabla 2. 2. Estructura de los datos asociados a un luchador

0	1	2	3	4	5	6
Identificador	Nivel	Puntos	Vida	Vida Máx	Ataque	Defensa

2.1.3 Diseño descendente

En este caso, el objetivo principal del problema planteado es ver cómo crear aplicaciones complejas de forma modular. Por tanto, no se llevará a cabo un diseño descendente completo hasta el último detalle, sino que éste servirá para realizar un esquema claro de cuáles son las acciones que debe llevar a cabo el programa y, en algunos casos, en qué orden. Por tanto, este apartado también tiene un papel de apoyo a la hora de presentarle el problema para que lo entienda.

De la descripción del problema general, podrían extraerse los subproblemas enumerados a continuación. Recuerde, sin embargo, que tal vez ésta no sea la única solución válida, es una propuesta de interpretación posible del enunciado. Puede haber otras descomposiciones válidas.

1. Generar los atributos del nuevo jugador.

2. Anunciar inicio del combate.

(a) Mostrar estado del jugador.

3. Elegir al adversario.

4. Combatir.

(a) Mostrar estado de los luchadores.

i. Mostrar estado del jugador.

ii. Mostrar estado del adversario.

(b) Elegir estrategia del jugador.

(c) Elegir estrategia del adversario.

(d) Resolver resultados de estrategias.

i. Tirar monedas.

ii. Penalizar a luchador.

iii. Dañar luchador.

iv. Curar luchador.

(e) Restaurar luchador.

5. Resolver resultado del combate.

(a) Otorgar puntuación.

(b) Subir de nivel.

(c) Finalización del juego.

Esta lista ya da una buena idea del conjunto de tareas a realizar. En este caso, a medida que se vaya resolviendo cada subproblema, si se considera que es todavía demasiado complejo o resulta que es un método demasiado largo, ya se harán nuevas descomposiciones en el mismo momento. Ésta es una estrategia aceptable para programas complejos, ya que la descomposición puede hacerse muy complicada, al ser difícil ver realmente todos los detalles y tener una idea clara del tamaño o complejidad de los métodos resultantes. Pero al menos, siempre debe tener la disciplina de hacer una primera aproximación, para generar el código fuente con una idea clara de por dónde empezar.

Antes de seguir, vale la pena hacer algunos comentarios. Los subproblemas 4.II y 4.III se han considerado diferentes ya que, si se fija, tendrán que llevar a cabo tareas bastante diferentes. En el caso del jugador, se pregunta directamente al usuario, mientras que en el caso del adversario el ordenador es quien debe generarlo de alguna manera (por ejemplo, simplemente al azar). En cambio, para el caso de los subproblemas 4.Ia yb, seguro que harán lo mismo. Sólo cambiarán los datos a tratar. Por tanto es un caso claro de parametrización de un método. Por último, este planteamiento también reaprovecha subproblemas, ya que el 4.Ia y el 2.I son exactamente el mismo.

2.1.4 Módulos

Si se quiere considerar una aproximación modular, una vez se conocen las tareas que debe llevar a cabo el programa en forma de subproblemas, el siguiente paso sería agruparlas de acuerdo al tipo de acciones que llevan a cabo. Cada conjunto será un módulo distinto. En este caso, dado que el programa se realiza en Java, ya se utilizará directamente una organización en clases y packages.

Como punto de partida, debería elegir un nombre de package general para todo el programa. Éste será `joc.arena`. La clase principal irá aquí.

A continuación, elegir si se quiere usar una jerarquía de packages que parta de la base para ordenar todas las clases o no. Para este caso, suele ser una buena política dividir a las partes vinculadas con la interfaz de usuario de las que están ligadas a la manipulación de los datos del programa. Al separar los aspectos relacionados con la presentación de los datos de su tratamiento, los cambios en los módulos de un programa (por ejemplo, pasar de una interfaz textual a una gráfica) no afectan al código de los módulos del otro. Esta división se puede realizar usando dos packages: `joc.arena.reglas` y `joc.arena.interficie`.

Ahora es el momento de dividir las tareas que debe realizar el programa en clases y elegir a qué package irá cada una. Este proceso es relativamente subjetivo y se aviene a la visión que tiene el programador sobre cómo estructurar su programa. En este sentido, la decisión es tan personal como decidir cómo clasificar fotos en carpetas dentro de un ordenador. De todas formas, en este proceso, lo que debe tener siempre en cuenta es que el objetivo final es organizar su código de forma que sea fácil de identificar dónde encontrar cada método.

Al aplicar modularidad, es necesario que cada clase encapsule un conjunto de tareas claramente relacionadas, independientemente del número de métodos que al final signifique que hay en cada clase (muchos o pocos).

El objetivo no es distribuir uniformemente los métodos para dar clases de tamaño similar. Si en un programa se identifican 30 métodos, el objetivo no es crear 6 clases para que haya 5 métodos en cada una.

Para este problema se propone la siguiente división en módulos. En el package `joc.arena.reglas` estarán las clases:

- Monedas: para las tareas vinculadas al lanzamiento de monedas por resolver una ronda.
- Luchador: para las tareas vinculadas a la manipulación de los datos de un luchador (dañar, curar, etc.).
- Bestiario: para las tareas vinculadas a la generación de adversarios y el jugador.

- Combate: para las tareas vinculadas a la resolución de estrategias enfrentamiento.

En el package `joc.arena.interficie` se decide dividir las clases que tratan la pantalla y el teclado, por lo que habrá:

- EntradaTeclado: se encarga de las tareas importantes que son dadas por lo que escribe el usuario usando el teclado.
- SalidaPantalla: como la anterior, pero para mostrar información en pantalla.

La solución completa del ejemplo empleado en este apartado se puede encontrar en la sección "Anexos" de la web

La clase principal, `JocArena` está en el package que engloba los anteriores, `joc.arena`, dada la jerarquía de nombres.

Una vez que se disponga de esta división, cada vez que sea necesario implementar un subproblema en forma de método, habrá que hacerlo en la clase que corresponda de acuerdo a esta distribución de tareas.

2.2 La biblioteca "juego.arena.reglas"

Antes de poder mostrar datos por pantalla, es necesario poder disponer de ellos y haberlos manipulado. Por tanto, lo que tiene más sentido es empezar por este package y no por `joc.arena.interficie`. De hecho, seguro que desde las clases para mostrar o entrar datos en el programa se invocarán métodos de tratamiento de datos. O sea, métodos de clases de ese package.

Por tanto, el primer package a tratar es éste.

2.2.1 La clase Monedas

Esta clase agrupa a los métodos vinculados a los aspectos aleatorios cuando se resuelve un combate. Básicamente, esto se reduce al lanzamiento de un cierto número de monedas para contar cuántas caras se han quitado. Esto puede llevarse a cabo usando la clase `Random`, perteneciente al package `java.util` de la API de Java, que permite generar valores al azar. Puesto que sólo se quiere mirar si se quita cara o cruz, se puede usar el método `nextBoolean`, por lo que si se evalúa `true`, se considera cara, y de lo contrario, cree. Dentro de ese programa, sería la clase más sencilla.

Como puede verse en el código fuente, esta clase sólo dispone de un método, ya que, dado el planteamiento del problema, sólo existe una acción vinculada al lanzamiento de monedas. Esto no es ningún problema. Recuerde que, para generar los módulos de un programa, el criterio principal y más importante es establecer parcelas diferenciadas según las funcionalidades o temáticas de los métodos incluidos. No se trata de intentar distribuirlos equitativamente en partes iguales.

```
1 package juego.arena.reglas;
2 importe java.util.Random;
3 public class Monedas {
4     /** Resuelve el lanzamiento de un grupo de monedas.
5     *
6     * @param numMonedas Número de monedas que se han lanzado.
7     * @return Número de caras obtenidas.
8     */
9     public int ferTirada(int numMonedas) {
10         Random rnd = new Random();
11         int numCares = 0;
12         for (int i = 0; i < numMonedas; i++) {
13             boolean tirón = rnd.nextBoolean();
14             if (tirón) {
15                 numCares++;
16             }
17         }
18         return numCares;
19     }
20 }
```

2.2.2 La clase Luchador

Esta clase es la más importante, ya que es la que gestiona la manipulación de el estado de los luchadores. Y para la descripción del problema, se puede ver que a uno luchador le pueden pasar muchas cosas... Por tanto, esta clase se tratará con mucho detalle, ya que sirve para aprender muchas cosas sobre cómo hacer un programa modular correcto.

Cuando es necesario tratar un conjunto de datos de forma compleja, siempre es interesante dedicar un módulo íntegramente a todas las operaciones que se quieran llevar a cabo con éstas. Sobre todo si algunas de las operaciones se usarán en muchos sitios diferentes. Un ejemplo es el caso de disponer de un repositorio de métodos para manipular arrays o cadenas de texto que siguen un formato especial. De este modo, estas manipulaciones quedan encapsuladas bajo el identificador de un método y el código es más fácil de seguir.

En este caso, existe un conjunto de datos muy particulares sobre los que es necesario realizar unas cuantas operaciones complejas: el array que representa cada luchador. Nada impide trabajar directamente con índices de posiciones para manipular los sus valores, pero es mejor crear un método para cada operación que se desea llevar a cabo. Es más fácil entender una instrucción donde pone dañar (jugador, 3) que no jugador[3] = jugador[3] - 2;. Además, muchas de ellas operaciones ya surgen de la descomposición del problema general (por ejemplo, subproblemas “Combatir” Resolver resultados de estrategias: Tirar monedas, penalizar luchador, dañar luchador, curar luchador”.

La estrategia final de esta clase es: si un método tendría como parámetro uno luchador, ya que debe llevar a cabo una tarea según sus valores, entonces va dentro de esta clase.

Métodos básicos de manipulación de datos

De acuerdo con la descripción del problema, al menos es necesario poder realizar las operaciones siguientes sobre un luchador, ya que modifican su estado:

- Dañar: restar puntos de vida.
- Curar: incrementar puntos de vida, hasta un máximo.
- Penalizar: restar puntos de Ataque o Defensa al azar, pero nunca de forma que el valor final quede por debajo de 1.
- Restaurar: recuperarse de las penalizaciones (se hace cada ronda múltiple de 5).
- Renovar: recuperar todos los puntos de vida y eliminar las penalizaciones (en subir de nivel).
- Otorgar puntuación: sumar puntos ganados por un combate.
- Subir de nivel: realizar el proceso de incremento de un nivel.

Una vez llegados a este punto, se puede detectar que para realizar una operación de restaurar, es necesario saber cuál era el valor original del Ataque o la Defensa. Por tanto, el planteamiento inicial de cómo estructurar el array con los datos de los luchadores no es suficiente. El hecho de encontrarse que debe reenfocar parte del diseño cuando ha llegado a la implementación del código no es algo extraño. La tabla 2.3 muestra el nuevo formato empleado para representar a un luchador.

Tabla 2. 3. Nueva estructura de los datos asociados a un luchador

0	1	2	3	4	5	6	7	8
Identificador	Nivel	Puntos	Vida	Vida Máx	Ataque	Ataque Máx	Defensa	Defensa Máx

Hay otro aspecto que vale la pena considerar en este caso, dónde cada posición del array que representa a un luchador tiene un significado muy concreto. Se trata de no usar directamente números para acceder a las posiciones del array, sino hacerlo en partir de constantes. Esto tiene dos funciones. Por un lado, mejora la legibilidad del programa. Por otra parte, si en el futuro se quieren incluir más datos en el array o cambiar alguno de los campos existentes (por ejemplo, ahora el nivel estará en la posición 3), sólo será necesario modificar el valor de la constante y el cambio se propagará al resto del código. Como acaba de ver, tener que cambiar el planteamiento de las posiciones de un array puede pasar perfectamente. Es necesario ser previsores.

Finalmente, para aprovechar la capacidad que le proporciona un programa para llevar a cabo cálculos de cierta complejidad, el otorgamiento de los puntos tendrá una particularidad.

Sobre los puntos que vale cada adversario se aplica un factor de corrección de 0*5 puntos por cada nivel de diferencia. De esta forma, adversarios proporcionalmente más difíciles valen más puntos, pero luchar repetidas veces contra adversarios de nivel demasiado bajo no reporta ningún punto. Para verlo, estudie el método otorgarPuntos.

```
1 package juego.arena.reglas;
2 importe java.util.Random;
3 public class Luchador {
4     //Formato
5     //Nombre:Nivel:XP:PV:Max PV:Ataque:Max Ataque:max Defensa
6     public final static int ID = 0;
7     public final static int NIVEL = 1;
8     public final static int PUNTOS = 2;
9     public final static int VIDA = 3;
10    public final static int VIDA_MAX = 4;
11    public final static int ATAC = 5;
12    public final static int ATAC_MAX = 6;
13    public final static int DEFENSA = 7;
14    public final static int DEFENSA_MAX = 8;
15    /** Inflige daño a un luchador, restando puntos de vida, hasta un mínimo de
16        0.
17        *
18        * @param luchador Luchador que recibe el daño
19        * @param puntos Puntos de vida que pierde
20        */
21    public void dañar(int[] luchador, int puntos) {
22        if (luchador[VIDA] > puntos) {
23            luchador[VIDA] = luchador[VIDA] - puntos;
24        } else {
25            luchador[VIDA] = 0;
26        }
27    }
28    /** Cura a un luchador, que recupera puntos de vida. Nunca puede superar
29        * lo máximo posible.
30        *
31        * @param luchador Luchador a curar
32        * @param puntos Puntos de vida recuperados
33        */
34    public void curar(int[] luchador, int puntos) {
35        luchador[VIDA] = luchador[VIDA] + puntos;
36        if (luchador[VIDA] > luchador[VIDA_MAX]) {
37            luchador[VIDA] = luchador[VIDA_MAX];
38        }
39    }
40    /** Aplica una penalización al luchador. Se hace al azar entre el valor
41        * de ataque y el de defensa. Se le resta un punto, hasta un valor mínimo de 1.
42        *
43        * @param luchador Luchador a penalizar
44        * @param grado Grado de penalización
45        */
46    public void penalizar(int[] luchador, int grado) {
47        //Se elige qué habilidad penalizar
48        Random rnd = new Random();
49        int penalizar = ATAC;
50        if (rnd.nextBoolean()) {
51            penalizar = DEFENSA;
52        }
53        //Se penaliza. Mínimo baja hasta 1
54        luchador[penalizar] -= grado;
55        if (luchador[penalizar] < 1) {
56            luchador[penalizar] = 1;
57        }
58    }
59    /** Renueva a un luchador, anulando todas las penalizaciones y daños.
60        *
61        * @param luchador Luchador a renovar
62        */
63    public void renovar(int[] luchador) {
64        restaurar(luchador);
65        luchador[VIDA] = luchador[VIDA_MAX];
66    }
67    /** Restaura los valores de ataque y defensa del luchador a los valores originales.
68        *
69        * @param luchador Luchador a restaurar
```

```

69 */
70 public void restaurar(int[] luchador) {
71     luchador[ATAC] = luchador[ATAC_MAX];
72     luchador[DEFENSA] = luchador[DEFENSA_MAX];
73 }
74 /** Resuelve el otorgamiento de puntos al aventurero al derrotar a un adversario. La
75  * cantidad de puntos depende de la diferencia de niveles entre los dos. Si se
76  * ganan suficientes puntos, se avisa si es necesario subir de nivel.
77  *
78  * @param aventurero Aventurero
79  * @param adversario Adversario derrotado
80  * @returns Si se ha subido de nivel (cada 100 puntos)
81  */
82 public boolean otorgarPuntos(int[] aventurero, int[] adversario) {
83     //Se calcula el multiplicador
84     float multiplicador = 0;
85     int numMultiplicadores = adversario[NIVEL] - aventurero[NIVEL] + 2;
86     for (int i = 0; i < numMultiplicadores; i++) {
87         multiplicador += 0.5;
88     }
89     //Puntos finales a otorgar
90     int puntosAdversario = leerPuntos(adversario);
91     int puntosOtorgados = Math.round(puntosAdversario*multiplicador);
92     //Sube de nivel?
93     aventurero[PUNTOS] += puntosOtorgados;
94     int nuevoNivel = 1 + aventurero[PUNTOS]/100;
95     if (nuevoNivel > aventurero[NIVEL]) {
96         return true;
97     }
98     return false;
99 }
100 /** Resuelve un incremento de un nivel, aumentando al azar ataque o defensa y dos
101  * puntos de vida máximos. Además, el luchador se cura totalmente.
102  *
103  * @param luchador Luchador que sube de nivel.
104  */
105 public void subirNivel(int[] luchador) {
106     luchador[NIVEL]++;
107     Random rnd = new Random();
108     if (rnd.nextBoolean()) {
109         //Se incrementa ataque
110         luchador[ATAC_MAX]++;
111     } else {
112         //Se incrementa defensa
113         luchador[DEFENSA_MAX]++;
114     }
115     luchador[VIDA_MAX] += 2;
116     //Se deja nueva de trunca
117     renovar(luchador);
118 }
119 }

```

Métodos vinculados al estado del luchador

Dado que esta clase incluye todos los métodos que dependen del estado de un luchador para hacer su trabajo, también es necesario incluir, no sólo los que modifican su estado, sino también los métodos cuyo resultado depende de ese estado. Estos incluyen los que realizan las siguientes operaciones:

- Calcular el grado de éxito de Ataque, ya que depende del valor de Ataque del luchador.
- Lo mismo para la defensa.
- Elegir una estrategia al azar, ya que se puede usar el estado del luchador para

tomar ciertas decisiones. Aquí se hará que si los puntos de vida del adversario son muy bajos, es más probable que decida defenderse.

```
1 //Métodos de acciones vinculadas al estado del luchador
2  /** Resuelve una tirada de ataque de un luchador. Se tiran tantas monedas como
3     * su valor de ataque.
4     *
5     * @param luchador Luchador que hace la tirada
6     * @return El número de caras obtenidas
7     */
8  public int tirarAtaque(int[] luchador) {
9      Monedas monedas = new Monedas();
10     return monedas.ferTirada(luchador[ATAQUE]);
11 }
12 /** Resuelve una tirada de defensa de un luchador. Se tiran tantas monedas como
13     * su valor de defensa.
14     *
15     * @param luchador Luchador que hace la tirada
16     * @return El número de caras obtenidas
17     */
18  public int tirarDefensa(int[] luchador) {
19      Monedas monedas = new Monedas();
20     return monedas.ferTirada(luchador[DEFENSA]);
21 }
22 /** Dado un luchador, elige al azar qué estrategia usar en una
23     * ronda de combate.
24     *
25     * @param luchador Luchador que elige la acción
26     * @return Acción elegida
27     */
28  public int elegirEstrategiaAzar(int[] luchador) {
29      Random rnd = new Random();
30      int limitDefensa = 3;
31      //Si le queda poca vida, defiende el 50% de los golpes
32      if (luchador[VIDA] < 2) {
33          limitDefensa = 1;
34      }
35      int accio = rnd.nextInt(10);
36      if ((acción >= 0)&&(acción < limitDefensa)) {
37          return Combat.ATAQUE;
38      } else if ((limitDefensa >= 3)&&(acción < 6)) {
39          return Combat.DEFENSA;
40      } else if ((acción >= 6)&&(acción < 8)) {
41          return Combat.ENGANY;
42      } else {
43          return Combat.MANIOBRA;
44      }
45 }
```

Métodos para facilitar la lectura de los datos

Por último, cuando se trabaja con conjuntos de datos con una función muy especial, como es este caso, puede valer la pena también incluir métodos que sirvan como atajos para hacer lecturas de los datos que contienen. Estos no hacen nada especial que no se podría hacer accediendo directamente al array por índice, pero pueden hacerlo código del resto de clases más esclarecedor, y además, en caso de documentar la clase usando el javadoc, sirven como una lista de los datos a los que se puede acceder individualmente.

```
1 //Métodos para facilitar la lectura de datos
2
3 /** Dice el identificador de un luchador. Usar un método facilita la lectura
4  * del código, más que acceder a posiciones de un array.
5  *
6  * @param luchador Luchador de quien se quiere leer el identificador
7  * @return Identificador del luchador
8  */
9 public int leerId(int[] luchador) {
10     return luchador[ID];
11 }
12 /** Dice cuál es el nivel de luchador. Usar un método facilita la lectura
13  * del código, más que acceder a posiciones de un array.
14  *
15  * @param luchador Luchador de quien se quiere leer el nivel
16  * @return
17  */
18 public int leerNivel(int[] luchador) {
19     return luchador[NIVEL];
20 }
21 /** Dice qué puntos vale el luchador. Usar un método facilita la lectura
22  * del código, más que acceder a posiciones de un array.
23  *
24  * @param luchador Luchador de quien se quieren leer los puntos
25  * @return
26  */
27 public int leerPuntos(int[] luchador) {
28     return luchador[PUNTOS];
29 }
30 /** Dice qué vida tiene el luchador. Usar un método facilita la lectura
31  * del código, más que acceder a posiciones de un array.
32  *
33  * @param luchador Luchador de quien se quiere leer la vida
34  * @return Vida
35  */
36 public int leerVida(int[] luchador) {
37     return luchador[VIDA];
38 }
39 /** Dice qué vida máxima tiene el luchador. Usar un método facilita la lectura
40  * del código, más que acceder a posiciones de un array.
41  *
42  * @param luchador Luchador de quien se quiere leer la vida máxima
43  * @return Vida
44  */
45 public int leerVidaMax(int[] luchador) {
46     return luchador[VIDA_MAX];
47 }
48 /** Dice qué ataque tiene el luchador. Usar un método facilita la lectura
49  * del código, más que acceder a posiciones de un array.
50  *
51  * @param luchador Luchador de quien se quiere leer el ataque
52  * @return Grado de ataque
53  */
54 public int leerAtaque(int[] luchador) {
55     return luchador[ATAC];
56 }
57 /** Dice qué defensa tiene el luchador. Usar un método facilita la lectura
58  * del código, más que acceder a posiciones de un array.
59  *
60  * @param luchador Luchador de quien se quiere leer la defensa
61  * @return Grado de defensa
62  */
63 public int leerDefensa(int[] luchador) {
64     return luchador[DEFENSA];
65 }
66 /** Dice si un luchador está muerto o no. O sea, si sus puntos de vida
67  * son 0 ahora mismo. Usar un método facilita la lectura del código, más que
68  * acceder a posiciones de un array.
69  *
70  * @param luchador Luchador a comprobar
```

```
70 * @return Si se considera muerto (true) o no (false)
71 */
72 public boolean esMort(int[] luchador) {
73     return (luchador[VIDA] == 0);
74 }
```

2.2.3 La clase Bestiario

Esta clase se encarga de todos los aspectos vinculados a la generación de los luchadores, tanto de los datos iniciales del jugador como el de los adversarios elegidos por el jugador (ver si hay lo que se ha pedido, y en su defecto, elegirlo al azar).

En este caso, se ha escogido que los adversarios no se generen al azar, sino que sus datos ya existan dentro del código fuente, por lo que, dado un adversario con un nombre concreto, siempre sea igual. Para almacenarlos, se usa un array bidimensional: un array donde en cada posición se encuentra el array que describe los valores de un adversario. Dado que es donde están definidos los nombres de los luchadores, también se gestiona la traducción de los identificadores de los luchadores a su nombre.

En esta clase también se muestra una posibilidad cuando se usan métodos de otra clase a menudo dentro del código fuente. En lugar de inicializar constantemente la clase por poder invocar sus métodos, existe la opción de declarar la variable donde se lleva a término la inicialización como global. Como variable global tiene un ámbito igual a toda la clase, puede ser usada desde cualquier sitio para invocar métodos de la otra clase. Esto es apenas lo que ocurre con la variable `luchador`. Fijese.

```
1 package juego.arena.reglas;
2 importe java.util.Random;
3 public class Bestiario {
4     //Mesa de traducción de identificadores a nombres
5     private String[] nombres = {"Aventurero",
6                                 "Nan", "Kobold",
7                                 "Orc", "Profundo",
8                                 "Brujo maligno", "Ogro",
9                                 "Guerrero del caos", "Troll",
10                                "Elemental terrestre", "Hidra"};
11
12     //Jugador: ID = 0
13     private int[] jugador = {0, 1, 0, 10, 10, 3, 3, 3, 3};
14     //Adversarios posibles en el juego
15     private int[][] adversarios = {
16         {1, 1, 25, 8, 8, 3, 3, 3, 3},
17         {2, 1, 30, 10, 10, 4, 4, 2, 2},
18         {3, 2, 35, 12, 12, 4, 4, 3, 3},
19         {4, 2, 40, 14, 14, 3, 3, 4, 4},
20         {5, 3, 45, 15, 15, 3, 3, 5, 5},
21         {6, 3, 50, 16, 16, 5, 5, 2, 2},
22         {7, 4, 55, 15, 15, 4, 4, 4, 4},
23         {8, 4, 60, 18, 18, 3, 3, 5, 5},
24         {9, 5, 70, 22, 22, 4, 4, 6, 6},
25         {10, 5, 80, 30, 30, 8, 8, 2, 2}
26     };
27
28     //Inicialización usando una variable global
29     private Luchador luchador = new Luchador();
30     /** Genera un nuevo jugador
31     *
32     * @return Un array con los datos de un jugador inicial
33     */
34     public int[] generarJugador() {
```

```

33     luchador.renovar(jugador);
34     return jugador;
35 }
36 /** Dado un nombre, genera el adversario correspondiente. Si ese nombre no existe,
37  * se genera al azar.
38  *
39  * @param nombreAdv Nombre del adversario a obtener
40  * @return El luchador con este nombre, o null si no existe
41  */
42 public int[] buscarAdversario(String nombreAdv) {
43     for (int i = 0; i < adversarios.length; i++) {
44         int id = luchador.leerId(adversarios[i]);
45         String nombre = traducirIDANom(id);
46         if (nombre.equalsIgnoreCase(nombreAdv)) {
47             luchador.renovar(adversarios[i]);
48             return adversarios[i];
49         }
50     }
51     return null;
52 }
53 /**Dado un nivel, genera el adversario correspondiente al azar. Se trata
54  * de un adversario que sea de este nivel al menos.
55  *
56  * @param nivel Nivel cercano al del adversario a obtener
57  * @return Un adversario
58  */
59 public int[] elegirAdversarioAzar(int nivel) {
60     Random rnd = new Random();
61     int[] adversario = null;
62     boolean buscar = true;
63     while (buscar) {
64         int y = rnd.nextInt(adversarios.length);
65         adversario = adversarios[y];
66         int nivelAdv = luchador.leerNivel(adversario);
67         int dif = nivel - nivelAdv;
68         if ((dif >= -1) && (dif <= 1)) {
69             buscar = false;
70         }
71     }
72     //Se deja al adversario nuevo de trinca, listo para luchar
73     luchador.renovar(adversario);
74     return adversario;
75 }
76 /** Transforma un identificador de luchador a su nombre.
77  *
78  * @param id Identificador
79  * @return La cadena de texto con el nombre.
80  */
81 public String traducirIDANom(int id) {
82     if ((id >= 0) && (id < nombres.length)) {
83         return nombres[id];
84     }
85     return "DESCONOCIDO";
86 }
87 }

```

2.2.4 La clase Combate

Esta clase se encarga de los aspectos vinculados a la resolución de una ronda de combate, dado el estado actual de los dos luchadores (puntos de vida, Ataque y Defensa) y la estrategia elegida por esta ronda. Básicamente, sobre ella recae el peso del subproblema “Combatir>Resolver resultados de estrategia” resultante del diseño

descendente. Puesto que éste todavía se divide en otros subproblemas, por hacer sus tareas tendrá que realizar invocaciones sobre otros métodos (concretamente, de la clase Luchador).

Para indicar cada estrategia se usa un valor entero, por lo que hacer comparaciones es mucho más sencillo que con cadenas de texto. Para facilitar la lectura del código, el valor asociado a cada estrategia se asigna a una constante. Fíjese cómo mediante invocaciones a métodos de la clase Luchador se modifica el estado de cada luchador según la resolución de la ronda. Si se han elegido los nombres de los métodos de forma adecuada, este sistema deja muy claro qué sucede en cada caso sin tener que ir a inspeccionar el código fuente de Luchador.

Además, con ya cierta previsión de cara a mostrar datos por pantalla, se incluye un método que sirve para transformar una acción dada a su representación en formato texto. Se trata del método estrategiaAText.

El código de esta clase sería el siguiente:

```
1 package juego.arena.reglas;
2 public class Combate {
3     //Constantes que indican posibles acciones de combate
4     public static final int ATAC = 0;
5     public static final int DEFENSA = 1;
6     public static final int ENGAÑO = 2;
7     public static final int MANIOBRA = 3;
8     /** Dado el código de una estrategia, lo convierte a un texto.
9     *
10    * @param acción Código de la estrategia
11    * @return Texto asociado
12    */
13    public String estrategiaAText(int accio) {
14        switch(acción) {
15            case ATAC: return "ATAC";
16            case DEFENSA: return "DEFENSA";
17            case ENGAÑO: return "ENGAÑO";
18            case MANIOBRA: return "MANIOBRA";
19        }
20        return "DESCONOCIDA";
21    }
22    /** Obtiene en grado de éxito correspondiente según la acción elegida por el luchador.
23    *
24    * @param ll Luchador que realiza la acción.
25    * @param accio Acción elegida
26    * @return Grado de éxito de la acción
27    */
28    public int calcularGrauExit(int[] ll, int accio) {
29        Luchador luchador = new Luchador();
30        switch(acción) {
31            case ATAQUE:
32            case ENGAÑO:
33                return luchador.tirarAtaque(ll);
34            default:
35                return luchador.tirarDefensa(ll);
36        }
37    }
38    /** Resuelve una ronda de acciones entre dos luchadores, de acuerdo con las estrategias
39    * individuales de cada uno.
40    *
41    * @param jug Estado del Jugador
42    * @param accioJug Estrategia elegida por el Jugador
43    * @param adv Estado del Adversario
44    * @param accioAdv Estrategia elegida por el Adversario.
45    */
```

```
46     public void resolverEstrategias(int[] jug, int accioJug, int[] adv, int
        accioAdv) {
47         int exitJug = calcular GraveExit(jug, accioJug);
48         int exitAdv = calcularGrauExit(adv, accioAdv);
49         Luchador luchador = new Luchador();
50         if ((accioJug == ATAC)&&(accioAdv == ATAC)) {
51             //Jug y Adv: Dañado
52             luchador.dañar(jug, exitAdv);
53             luchador.dañar(adv, exitJug);
54         } else if ((accioJug == ATAC)&&(accioAdv == DEFENSA)) {
55             //Adv: Curado
56             luchador.curar(adv, exitAdv);
57         } else if ((accioJug == ATAC)&&(accioAdv == ENGAÑO)) {
58             //Adv: Dañado
59             luchador.dañar(adv, exitJug);
60         } else if ((accioJug == ATAC)&&(accioAdv == MANIOBRA)) {
61             //Adv: Dañado
62             luchador.dañar(adv, exitJug);
63         } else if ((accioJug == DEFENSA)&&(accioAdv == ATAC)) {
64             //Jug: Curado
65             luchador.curar(jug, exitJug);
66         } else if ((accioJug == DEFENSA)&&(accioAdv == DEFENSA)) {
67             //Jug y Adv: Curado
68             luchador.curar(adv, exitAdv);
69             luchador.curar(jug, exitJug);
70         } else if ((accioJug == DEFENSA)&&(accioAdv == ENGAÑO)) {
71             //Jug: Dañado x2
72             luchador.dañar(jug, exitAdv*2);
73         } else if ((accioJug == DEFENSA)&&(accioAdv == MANIOBRA)) {
74             //Jug: Penalizado
75             luchador.penalizar(jug, exitAdv);
76         } else if ((accioJug == ENGAÑO)&&(accioAdv == ATAC)) {
77             //Jug: Dañado
78             luchador.dañar(jug, exitAdv);
79         } else if ((accioJug == ENGAÑO)&&(accioAdv == DEFENSA)) {
80             //Adv: Dañado x2
81             luchador.dañar(adv, exitJug*2);
82         } else if ((accioJug == ENGAÑO)&&(accioAdv == ENGAÑO)) {
83             //Jug y Adv: Dañado
84             luchador.dañar(jug, exitAdv);
85             luchador.dañar(adv, exitJug);
86         } else if ((accioJug == ENGAÑO)&&(accioAdv == MANIOBRA)) {
87             //Jug: Penalizado
88             luchador.penalizar(jug, exitAdv);
89         } else if ((accioJug == MANIOBRA)&&(accioAdv == ATAC)) {
90             //Jug: Dañado
91             luchador.dañar(jug, exitAdv);
92         } else if ((accioJug == MANIOBRA)&&(accioAdv == DEFENSA)) {
93             //Adv: Penalizado
94             luchador.penalizar(adv, exitJug);
95         } else if ((accioJug == MANIOBRA)&&(accioAdv == ENGAÑO)) {
96             //Adv: Penalizado
97             luchador.penalizar(adv, exitJug);
98         } else if ((accioJug == MANIOBRA)&&(accioAdv == MANIOBRA)) {
99             //Jug y Adv: Penalizado
100             luchador.penalizar(adv, exitJug);
101             luchador.penalizar(jug, exitAdv);
102         } else {
103             //No debería darse este caso...
104         }
105     }
106 }
```

2.3 La biblioteca "juego.arena.interfaz"

Una vez se dispone del código fuente de todas las clases que gestionan los datos en manipular, ya es posible generar las que las obtienen o muestran al usuario.

2.3.1 La clase EntradaTeclado

Dada la descripción del problema general, sólo hay dos casos en los que el usuario debe entrar datos usando el teclado. Para indicar contra qué adversario quiere luchar en iniciarse un combate y por decir la estrategia a seguir en una ronda de combate. Por por tanto, sólo hay que incluir dos métodos:

- `elegirAdversario`, asociado al subproblema "Elegir al adversario".
- `preguntarEstrategia`, asociado al subproblema "Combatir>Elegir estrategia del jugador".

Para el caso de la estrategia, se dará a elegir con un menú con cuatro opciones, cada una asociada a una letra: [A]manchar, [D]efensar, [E]ngany y [M]aniobra.

Un hecho interesante de esta clase es que, dado que en la clase Combate las cuatro estrategias posibles están indicadas mediante constantes, es necesario traducir la letra que ha escrito el usuario en el valor de la constante asociada. Para acceder a una constante declarada a otra clase, es necesario usar la misma sintaxis que para invocar un método estático, pero usando el identificador de la contante: `NombreClasse.NOM_CONSTANT`. Esto se puede ver en el código del método `preguntarEstrategia`.

```
1 package juego.arena.interfaz;
2 importe java.util.Scanner;
3 importe juego.arena.reglas.Bestiario;
4 importe juego.arena.reglas.Combate;
5 public class EntradaTeclado {
6     /** Elige al adversario del jugador según su respuesta.
7     *
8     * @return Cadena de texto con la respuesta
9     */
10    public int[] elegirAdversario(int nivel) {
11        System.out.print("¿Contra qué adversario quieres luchar en este combate? ");
12        Scanner lector = new Scanner(System.in);
13        String respuesta = lector.nextLine();
14        Bestiario bestiario = new Bestiario();
15        int[] adversario = bestiario.buscarAdversario(respuesta);
16        if (adversario == null) {
17            System.out.println("Este enemigo no existe. Se elige al aztar.");
18            adversario = bestiario.elegirAdversarioAzar(nivel);
19        }
20        return adversario;
21    }
22    /** Pregunta al usuario qué estrategia quiere usar, de entre
23    * las cuatro posibles.
24    *
25    * @return Acción a llevar a cabo, de acuerdo a las constantes de la clase Combate.
26    */
```

```

27     public int preguntarEstrategia() {
28         Scanner lector = new Scanner(System.in);
29         System.out.println("¿Qué estrategia quieres seguir esta ronda?");
30         System.out.println("[A]manchar\t[D]efensar\t[E]engaño\t[M]aniobra");
31         System.out.println("-----");
32         boolean preguntar = true;
33         int accion = -1;
34         while (preguntar) {
35             System.out.print("Accio: ");
36             String respuesta = lector.nextLine();
37             if ("A".equalsIgnoreCase(respuesta)) {
38                 accion = Combat.ATAC;
39                 preguntar = false;
40             } else if ("D".equalsIgnoreCase(respuesta)) {
41                 accion = Combat.DEFENSA;
42                 preguntar = false;
43             } else if ("E".equalsIgnoreCase(respuesta)) {
44                 accion = Combate.ENGANO;
45                 preguntar = false;
46             } else if ("M".equalsIgnoreCase(respuesta)) {
47                 accion = Combate.MANIOBRA;
48                 preguntar = false;
49             } else {
50                 System.out.print("Acción incorrecta...");
51             }
52         }
53         return accion;
54     }
55 }

```

2.3.2 La clase SalidaPantalla

En esta clase se agruparían los métodos en los que se necesitan más de una única instrucción para mostrar información por pantalla. Concretamente se trataría de los métodos asociados a los subproblemas “Anunciar inicio de combates>Mostrar estado del jugador” y “Combatir>Mostrar el estado de los luchadores: Mostrado estado del jugador, Mostrar estado del adversario” del resultado de aplicar diseño descendente. de una clase muy compleja.

```

1 package juego.arena.interfaz;
2 importe juego.arena.reglas.Bestiario;
3 importe juego.arena.reglas.Luchador;
4 public class SalidaPantalla {
5     /** Muestra por pantalla el mensaje de inicio del Juego
6      *
7      */
8     public void mostrarBienvenida() {
9         System.out.println("Bienvenido al Juego de la Arena");
10        System.out.println("=====");
11        System.out.println("Elige con astucia tu estrategia para sobrevivir...");
12    }
13    /** Muestra el estado de un luchador por pantalla.
14     *
15     * @param ll Luchador a visualizar
16     */
17    public void mostrarLuchador(int[] ll) {
18        Luchador luchador = new Luchador();
19        Bestiario bestiario = new Bestiario();
20        int id = luchador.leerId(ll);
21        String nombre = bestiario.traducirIDANom(id);
22        System.out.print(nombre);

```

```

23     System.out.print("\tNivel: " + luchador.leerNivel(l));
24     System.out.print(" (puntos: " + luchador.leerPuntos(l) + ")");
25     System.out.print("\tVIDA: " + luchador.leerVida(l));
26     System.out.print(" (" + luchador.leerVidaMax(l) + ")");
27     System.out.print("\tATAC: " + luchador.leerAtaque(l));
28     System.out.println("\tDEFENSA: " + luchador.leerDefensa(l));
29 }
30 /** Muestra el estado actual del jugador contra su adversario.
31  *
32  * @param jugador Jugador
33  * @param adversario Adversario
34  */
35 public void mostrarVersus(int[] jugador, int[] adversario) {
36     System.out.print("JUGADOR: ");
37     mostrarLuchador(jugador);
38     System.out.println("VS");
39     System.out.print("ADVERSARIO: ");
40     mostrarLuchador(adversario);
41 }
42 }

```

2.4 La clase principal

Cuando se genera un programa modular estructurado de acuerdo a una cierta jerarquía de packages, la clase principal se suele ubicar en el package raíz, sola. Recuerde que ésta es la que se encarga de resolver el problema general a partir de la invocación de métodos de todo el resto de clases de los demás packages.

Para ese programa, su código sería el siguiente. Observe cómo, nuevamente, su estructura es muy cercana al primer nivel de descomposición del diseño descendente.

```

1 package juego.arena;
2 importe juego.arena.reglas.Bestiarío;
3 importe juego.arena.reglas.Combat;
4 importe juego.arena.reglas.Luchador;
5 importe juego.arena.interfaz.EntradaTeclado;
6 importe juego.arena.interfaz.SalidaPantalla;
7 public class JuegoArena {
8     public static final int MAX_COMBAT = 10;
9     private EntradaTeclado entrada = new EntradaTeclado();
10    private SalidaPantalla salida = new SalidaPantalla();
11    private Luchador luchador = new Luchador();
12    private Combat combate = new Combat();
13    private Bestiarío bestiarío = new Bestiarío();
14    public static void main(String[] args) {
15        JuegoArena programa = new JuegoArena();
16        programa.inicio();
17    }
18    public void inicio() {
19        salida.mostrarBienvenida();
20        int[] jugador = bestiarío.generarJugador();
21        int numCombat = 0;
22        boolean jugar = true;
23        while (jugar) {
24            numCombat++;
25            //Antes de cada combate se restaura al jugador
26            luchador.restaurar(jugador);
27            //Inicio de un combate
28            System.out.println("**** COMBAT" + numCombat);
29            System.out.print("Estado actual del jugador: ");

```

```
30     salida.mostrarLuchador(jugador);
31     System.out.println("*****");
32     //Se obtiene el adversario
33     int[] adversario = entrada.elegirAdversario(luchador.leerNivel(jugador))
34     ;
35     //Combate
36     combatir(jugador, adversario);
37     //Fi
38     jugar = fiCombat(jugador, adversario);
39     if (numCombat == MAX_COMBAT) {
40         System.out.println("Has sobrevivido a todos los combates. ¡¡¡Enhorabuena!!!");
41     }
42     System.out.print("Estado final del jugador: ");
43     salida.mostrarLuchador(jugador);
44 }
45 /** Resuelve todas las rondas de un combate.
46  *
47  * @param jugador Estado del jugador
48  * @param adversario Estado del adversario
49  */
50 public void combatir(int[] jugador, int[] adversario) {
51     boolean combatir = true;
52     int numRonda = 0;
53     while (combater) {
54         numRonda++;
55         if (numRonda%5 == 0) {
56             //En las rondas múltiples de cinco se restauran el ataque y la defensa
57             luchador.restaurar(jugador);
58             luchador.restaurar(adversario);
59         }
60         System.out.println("--- RONDA " + numRonda);
61         salida.mostrarVersus(jugador, adversario);
62         System.out.println("----- ");
63         int accioJug = entrada.preguntarEstrategia();
64         int accioAdv = luchador.elegirEstrategiaAzar(adversario);
65         System.out.print("Has elegido " + combat.estrategiaAText(accioJug));
66         System.out.println(" y tu enemigo " + combat.estrategiaAText(accioAdv)
67         );
68         combate.resolverEstrategias(jugador, accioJug, adversario, accioAdv);
69         if (luchador.esMuerte(jugador)||luchador.esMuerte(adversario)) {
70             combatir = false;
71         }
72     }
73     /** Resuelve la finalización del combate.
74     *
75     * @param jugador Estado del jugador
76     * @param adversario Estado del adversario
77     * @returns Si el jugador debe seguir jugando (true) o no (false)
78     */
79     public boolean fiCombat(int[] jugador, int[] adversario) {
80         if (luchador.esMuerte(jugador)) {
81             //Has perdido (Nota: también incluye el caso de que ambos mueren a la vez)
82             System.out.println("Has sido derrotado... :-(");
83             return false;
84         }
85         System.out.println("Has ganado el combate :-)");
86         boolean subirNivel = luchador.otorgarPuntos(jugador, adversario);
87         if (subirNivel) {
88             System.out.println("¡Has subido de nivel!!!");
89             luchador.subirNivel(jugador);
90         }
91         return true;
92     }
93 }
```

2.5 Simplificación de algoritmos complejos usando recursividad

A la hora de crear programas complejos, uno de los aspectos que diferencia al buen programador del aficionado es su capacidad de realizar algoritmos eficientes. O sea, que sean capaces de resolver el problema planteado en el mínimo paso.

En el caso de un programa, esto significa la necesidad de ejecutar el mínimo número de instrucciones posible. Ciertamente, si el resultado debe ser exactamente el mismo, siempre será mejor realizar una tarea en 10 pasos que en 20, intentando evitar pasos que en realidad son innecesarios. Por tanto, la etapa de diseño de un algoritmo es bastante importante y hay que pensar bien una estrategia eficiente. Sin embargo, normalmente, los algoritmos más eficientes también son más difíciles de pensar y codificar, ya que no siempre son evidentes.

Un ejemplo muy sencillo de esto es la resolución del siguiente problema. Suponga que una amiga apunta un número entre el 0 y el 99 en una hoja de papel y usted debe adivinarla. Cada vez que conteste, le dirá si el valor que ha dicho es mayor o menor que el que debe adivinar. ¿Qué estrategia seguiría para lograrlo?
Es necesario pensar un algoritmo a seguir para resolver este problema.

Una aproximación muy ingenua podría ser ir diciendo todos los valores uno por uno, empezando por el 0. Está claro que cuando llegue al 99 lo habrá adivinado. En el mejor caso, si había escrito el 0, acertará a la primera, mientras que en el peor caso, si había escrito el 99, necesitará 100 intentos. Si estaba de por medio, quizás con 40-70 es suficiente. Éste sería un algoritmo que hace el hecho y es muy sencillo, pero no muy eficiente. Ir probando valores al azar en lugar de hacer esto tampoco mejora mucho el proceso, y viene a ser el mismo.

Seguro que si alguna vez ha jugado a este juego, lo que ha hecho es ser un poco más astutos y empezar por algún valor del medio. En este caso, por ejemplo, podría ser el 50. Entonces, en caso de fallar, sabiendo si el valor secreto es mayor o menor que su respuesta, en el siguiente intento probar un valor más alto o más bajo, e ir haciendo esto repetidas veces.

Por lo general, la mejor estrategia para adivinar un número secreto entre 0 y N sería primero probar $N/2$. Si no se ha acertado, entonces si el número secreto es mayor se intenta adivinar entre $(N/2 + 1)$ y N. Si era más bajo, se intenta adivinar el valor entre 0 y $N-1$. Para cada caso, se vuelve a probar el valor que hay en medio del nuevo intervalo. Y así sucesivamente, haciendo cada vez menor el intervalo de búsqueda, hasta adivinarlo. En el caso de 100 valores, esto garantiza que, en el peor de los casos, en 7 intentos seguro que se adivina. Esto es una mejora muy grande respecto al primer algoritmo, donde se necesitaban 100 intentos, y por tanto, éste sería un algoritmo más eficiente. Concretamente, siempre se adivinará en $\log_2(N)$ intentos como máximo.

Si se fija, el ejemplo que acaba de explicarse, en realidad, no es más que un esquema de búsqueda dentro de una secuencia de valores, como puede ser dentro de un array, partiendo de la condición de que todos los elementos estén ordenados de menor a mayor. De hecho, hasta ahora, para realizar una búsqueda de un valor dentro de un array se ha usado el sistema "ingenu", mirando una por una todas las posiciones. Pero si los elementos están

El método `binarySearch` de la clase `Arrays` realiza una búsqueda dicotómica.

ordenados previamente, se podría usar el sistema "astuto" para diseñar un algoritmo mucho más eficiente, y hasta cierto punto, más "inteligente".

El algoritmo basado en esta estrategia se conoce como búsqueda binaria o dicotómico-es.

Por tanto, ¿por qué no aplicar este conocimiento para mejorar el método `buscarAdversario`, en la clase `Bestiario`?

2.5.1 Aplicación de la recursividad

Desgraciadamente, a menudo se encontrará que explicar de palabra la idea general de una estrategia puede ser sencillo, pero traducirla a instrucciones de Java ya no lo es tanto. Dado que hay que ir repitiendo unos pasos en sucesivas iteraciones, está más o menos claro que el problema planteado para realizar búsquedas eficientes se basa en una estructura de repetición. Pero no se recorren todos los elementos y el índice no se incrementa uno a uno, sino que se va cambiando a valores muy distintos por cada iteración. No es un caso evidente. Precisamente, este ejemplo no se ha elegido al azar, ya que es un caso en el que puede ir bien aplicar un nuevo concepto que permite facilitar la definición de algoritmos complejos donde hay repeticiones.

La recursividad es una forma de describir un proceso para resolver un problema de modo que, a lo largo de esta descripción, se usa el mismo proceso que se está describiendo, pero aplicado a un caso más simple.

De hecho, quizá sin darse cuenta, ya se ha usado recursividad para describir cómo resolver un problema. Para ver qué significa exactamente la definición formal apenas descrita, se repetirá el texto en cuestión, pero remarcando el aspecto recursivo de la descripción:



Una definición recursiva: las iniciales del sistema operativo GNU significa "GNU is Not Unix". Fuente: The GNU Art Gallery

"Generalmente, la mejor estrategia para adivinar un número secreto entre 0 y N sería primero probar $N/2$. Si no se ha acertado, entonces si el número secreto es más alto se intenta adivinar entre $(N/2 + 1)$ y N. Si era más bajo, se intenta adivinar el valor entre 0 y $N-1$. así sucesivamente, hasta adivinarlo."

O sea, ¡el proceso de adivinar un número se basa en el proceso de intentar adivinar un número! Esto parece hacer trampas, ya es como usar la misma palabra que se quiere definir en su propia definición. Pero fíjese en un detalle muy importante.

Los nuevos usos del proceso de "adivinar" son casos más simples, ya que primero se adivina entre N valores posibles, después entre $N/2$ valores, después entre $N/4$, etc. Este hecho no es casual y de él depende poder definir un proceso recursivo de forma correcta.

2.5.2 Implementación de la recursividad

La implementación de la recursividad dentro del código fuente de un programa se realiza a nivel de método.

Un método recursivo es aquél que, dentro de su bloque de instrucciones, tiene alguna invocación a sí mismo.

El bloque de código de un método recursivo siempre se basa en una estructura de selección múltiple, en la que cada rama es de alguno de los dos casos posibles descritos a continuación.

Por un lado, en el caso base, que contiene un bloque de instrucciones dentro de las cuales no hay ninguna llamada al método mismo. Se ejecuta cuando se considera que, a partir de los parámetros de entrada, el problema ya es lo suficientemente simple como para ser resuelto directamente. En el caso de la búsqueda, sería cuando la posición intermedia es exactamente el valor que se está buscando, o cuando ya se puede decidir que el elemento a buscar no existe.

Por otra parte, existe el caso recursivo, que contiene un bloque de instrucciones dentro de las cuales hay un llamamiento al propio método, dado que se considera que todavía no se puede resolver el problema fácilmente. Ahora bien, valores usados como parámetros de esta nueva llamada deben ser distintos a los originales. Concretamente, deben ser unos valores que tiendan a acercarse al caso base. En el caso de la búsqueda, se corresponde con la búsqueda sobre la mitad de los valores originales, ya sea hacia la mitad inferior o superior.

Éste es un caso en el que el intervalo de posiciones donde se hará la nueva búsqueda se va acercando al caso base, ya que tarde o temprano, llamada tras llamada, el espacio de búsqueda se irá reduciendo hasta que, o bien se encuentra el elemento, o queda claro que no está.

Dentro de la estructura de selección siempre debe haber al menos un caso base y uno recursivo. Normalmente, los algoritmos recursivos más sencillos tienen uno de cada uno. Es imprescindible que los casos recursivos siempre garanticen que sucesivos llamamientos van aproximando los valores de los parámetros de entrada a algún caso base, ya que, de lo contrario, el programa nunca termina y se produce el mismo efecto que un bucle infinito.

En Java, en caso de un bucle infinito al realizar llamadas recursivas se produce un error de Stack Overflow ("desbordamiento de pila", en inglés).

Cálculo recursivo de la operación factorial

Como ejemplo del funcionamiento de un método recursivo, se empezará con un caso sencillo. Se trata del cálculo de la llamada operación factorial de un valor entero positivo. Ésta es unaria y se expresa con el operador exclamación (por ejemplo, 4!, 20!, 3!). El resultado de esta operación es la multiplicación de todos los valores desde el 1 hasta el indicado ($7! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 \cdot 7$). Normalmente, la definición matemática de esta operación se hace de forma recursiva:

- $0! = 1$ (caso base)
- $n! = n \cdot (n - 1)!$ (caso recursivo)

Así pues, fíjese que el caso recursivo realiza un cálculo que depende de usar la propia definición de la operación, pero cuando lo hace es con un nuevo valor inferior a el original, por lo que se garantiza que, en algún momento, se hará un llamamiento recursiva que desembocará en caso base. Cuando esto ocurra, la cadena de llamadas recursivas termina. Una forma de ver esto es desarrollando paso a paso esta definición:

$$1. 4! = 4 \cdot (4 - 1)! = 4 \cdot (3)!$$

$$2. 4 \cdot 3! = 4 \cdot (3 \cdot (3 - 1))! = 4 \cdot 3 \cdot (2)!$$

$$3. 4 \cdot 3 \cdot 2! = 4 \cdot 3 \cdot (2 \cdot (2 - 1))! = 4 \cdot 3 \cdot 2 \cdot (1)!$$

$$4. 4 \cdot 3 \cdot 2 \cdot 1! = 4 \cdot 3 \cdot 2 \cdot (1 \cdot (1 - 1))! = 4 \cdot 3 \cdot 2 \cdot 1 \cdot (0)!$$

$$5. 4 \cdot 3 \cdot 2 \cdot 1 \cdot 0! = 4 \cdot 3 \cdot 2 \cdot 1 \cdot (1) = 24$$

Su implementación en Java sería la siguiente. Ahora bien, en este código se han añadido algunas sentencias para escribir información por pantalla, por lo que se vea con más detalle cómo funciona un método recursivo. Verá que, inicialmente, se llevan a cabo una serie de invocaciones del caso recursivo, uno tras otro, hasta que se llega a una llamada que ejecuta el caso base. Es a partir de entonces cuando, a medida que se van ejecutando las sentencias return del caso recursivo, realmente se va acumulando el cálculo. Otra forma de verlo es depurando el programa.

```

1 package unidad5.apartado2.ejemplos;
2
3 public class Factorial {
4
5     public static void main(String[] args) {
6         Factorial programa = new Factorial();
7         programa.inicio();
8     }
9
10    public void inicio() {
11        System.out.println(factorial(4));
12    }
13
14    /** Método recursivo que calcula la operación factorial
15     *
16     * @param n Operador
17     * @return Resultado de n!
18     */
19    public int factorial (int n) {
20        if (n == 0) {
21            //Caso base: Se sabe el resultado directamente
22            System.out.println("Caso base: Se evalúa a 0");
23            return 1;
24        } else {
25            //Caso recursivo: Para calcularlo es necesario invocar el propio método
26            //El valor del nuevo parámetro de entrada debe variar de forma que
27            //se vaya aproximando al caso base
28            System.out.println("Caso recursivo" + (n - 1) + ": Se invoca el factorial")
29                ;
30            int res = n*factorial(n - 1);
31            System.out.println("Caso recursivo" return nada; + (n - 1) + ": Resultado = " + nada);
32        }
33    }
34
35 }
```

Cálculo recursivo de la búsqueda dicotómica

A continuación se muestra el código del algoritmo recursivo de búsqueda dicotómica sobre un array. Observe atentamente los comentarios, que identifican los casos base y recursivos. En este caso, existen más de un caso base y recursivo. Si desea ver con más detalle cómo funciona, puede depurarlo para ver cómo van evolucionando los valores de los parámetros de entrada en sucesivas invocaciones en los casos recursivos.

```
1 package unidad5.apartado2.ejemplos;
2
3 public class BúsquedaDicotomica {
4
5     public static void main(String[] args) {
6         BuscarDicotomica programa = new BuscarDicotomica();
7         programa.inicio();
8     }
9
10    public void inicio() {
11        int[] array = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
12        int búsquedaDivuit = búsquedaDicotomica(array, 0, array.length-1, 18);
13        int búsquedaCinco = búsquedaDicotomica(array, 0, array.length-1, 5);
14        System.out.println("Búsqueda del 18: " + búsquedaDivuit);
15        System.out.println("Búsqueda del 5: " + búsquedaCinco);
16    }
17
18    /** Búsqueda dicotómica recursiva sobre un array de enteros.
19     *
20     * @param array Dónde se realiza la búsqueda
21     * @param inicio Posición inicial de la búsqueda
22     * @param fin Posición final
23     * @param valor Valor a buscar
24     * @return Índice dónde está el valor, o -1 si no existe
25     */
26    public int búsquedaDicotomica(int[] array, int inicio, int fin, int valor) {
27        if (inicio > fin) {
28            //Caso base: No se ha encontrado el valor
29            return -1;
30        }
31        //Se calcula la posición central entre los dos índices de búsqueda
32        int pos = inicio + (fin - inicio) / 2;
33        if (array[pos] > valor) {
34            //Caso recursivo: Si el valor es menor que la posición que se ha mirado
35            //entonces hay que seguir buscando por la parte "derecha" del array
36            return buscaDicotomica(array, inicio, pos - 1, valor);
37        } else if (array[pos] < valor) {
38            //Caso recursivo: Si el valor es mayor que la posición que se ha mirado
39            //entonces hay que seguir buscando por la parte "izquierda" del array
40            return buscaDicotomica(array, pos + 1, fin, valor);
41        } else {
42            //caso base: Da igual, por tanto, se ha encontrado
43            return pos;
44        }
45    }
46 }
```

Prácticamente cualquier problema que puede resolverse con un algoritmo recursivo también puede resolverse con sentencias de estructuras de repetición. Pero muy a menudo su implementación será mucho menos evidente y las interacciones entre instrucciones bastante más complejas que la opción recursiva (una vez se entiende este concepto, es claro).

2.5.3 Recursividad al juego de combates en la arena

Una vez que conozca la base genérica para generar un algoritmo recursivo, es el momento de aplicarlo para incluir una búsqueda dicotómica dentro del juego propuesto. En este caso, se trataría de mejorar el método `buscarAdversario`, por lo que, en sitio de hacer la búsqueda posición por posición, se haga de forma más eficiente. La búsqueda se hace sobre los nombres de los adversarios, que deben estar previamente ordenados alfabéticamente. Por tanto, para poder llevar a cabo esta mejora, es necesario garantizar que la lista de nombres está ordenada y que los valores de los identificadores en los arrays con los atributos de todos los adversarios siguen encajando con la nueva lista de nombres. Hacer esto manualmente puede ser algo pesado, sobre todo si en el futuro se añaden nuevos adversarios. La ordenación se puede llevar a cabo con código justo al principio del programa.

Por las características modulares del programa, el nuevo método de búsqueda y el de ordenación irían dentro de la clase `Bestiario`. A continuación se muestra el nuevo código que es necesario añadir. En el caso de `buscarAdversario`, éste reemplazaría el ya existente. El resto son nuevos.

Estudie atentamente el siguiente código, que presenta la solución a lo que se plantea. Preste especial atención a cómo se hace la comparación entre cadenas de texto mediante el método `compareTo` de la clase `String`. Éste indica si una cadena de texto da igual, o tiene un orden alfabético inferior o superior a otra. También tenga en cuenta que las mayúsculas y minúsculas se consideran letras diferentes y afectan el orden de las palabras. Por tanto, para evitar que esto suceda, siempre se pasa todo a minúscula antes de comparar, con el método `toLowerCase`.

```
1 //Métodos por recursividad en la clase Bestiario
2 /** Dado el array bidimensional con los adversarios, los ordena por el nombre
3  * asociado a cada identificador. Recordar que un array bidimensional es
4  * considera un "array de arrays"
5  */
6 public void ordenarAdversarios() {
7     for (int i = 0; i < adversaris.length; i++) {
8         for (int j = i + 1; j < adversaris.length; j++) {
9             int idI = luchador.leerId(adversarios[i]);
10            String nombrel = traducirIDANom(idI);
11            nombrel = nombrel.toLowerCase();
12            int idJ = luchador.leerId(adversarios[j]);
13            String nombreJ = traducirIDANom(idJ);
14            nombreJ = nombreJ.toLowerCase();
15            if (nombrel.compareTo(nombreJ) > 0) {
16                //I > J. Hay que intercambiar
17                int tamañoArray = adversarios[i].length;
18                int[] temp = new int[tamañoArray];
19                copiarArray(adversarios[i], temp);
20                copiarArray(adversarios[j], adversarios[i]);
21                copiarArray(tempo, adversarios[j]);
22            }
23        }
24    }
25 }
26 /** Método auxiliar para copiar los valores de un array a otro.
27  *
28  * @param origen Array origen
29  * @param destino Array destino
30  */
```

```

31 public void copiarArray(int[] origen, int[] desti) {
32     for (int i=0; y < desti.length; i++) {
33         destino[i] = origen[i];
34     }
35 }
36 /** Inicio de la búsqueda recursiva. Adapta la llamada normal de acuerdo al tamaño
37  * de la lista de adversarios.
38  *
39  * @param nombreAdv Nombre a buscar
40  * @return Adversario (null si no se encuentra)
41  */
42 public int[] buscarAdversario(String nombreAdv) {
43     return buscarAdversarioRecursivamente(nombreAdv.toLowerCase(), 0, adversarios.
44         length - 1);
45 }
46 /** Búsqueda recursiva real
47  *
48  * @param nombreAdv Nombre a buscar
49  * @param inicio Posición inicial de la búsqueda
50  * @param fin Posición final de la búsqueda
51  * @return Adversario encontrado (o null si no se encuentra)
52  */
53 public int[] buscarAdversarioRecursivamente(String nombreAdv, int inicio, int fin) {
54     int diferencia = fin - inicio;
55     if (diferencia < 0) {
56         //No se ha encontrado
57         return null;
58     } else {
59         int posición = inicio + diferencia/2;
60         int id = luchador.leerId(adversarios[posición]);
61         String nombre = traducirIDANom(id);
62         nombre = nombre.toLowerCase();
63
64         int compara = nombre.compareTo(nombreAdv);
65         if (compara == 0) {
66             //Son iguales. Encontrado!
67             return adversarios[posición];
68         } else if (compara < 0) {
69             //El nombre de la lista es más pequeño. Hay que buscar por los posteriores
70             return buscarAdversarioRecursivamente(nombreAdv, posición + 1, fin);
71         } else {
72             //El nombre de la lista es mayor. Es necesario buscar por los anteriores
73             return buscarAdversarioRecursivamente(nombreAdv, inicio, posición - 1);
74         }
75     }
76 }

```

Para garantizar que los nombres están ordenados alfabéticamente, será necesario llamar al método de ordenación en cuanto se inicie el programa, en el método inicio en la clase JuegoArena.

```

1 public void inicio() {
2     //PARTE RECURSIVA
3     bestiario.ordenarAdversarios();
4     //De hecho, fuera de la ordenación, todo el código se mantiene idéntico
5     salida.mostrarBienvenida();
6     int[] jugador = bestiario.generarJugador();
7     //etc.
8 }

```

Con estas enmiendas, se ha aplicado recursividad para mejorar la eficiencia del programa. Claro que, para encontrar adversarios en una lista de 10 elementos no será paso evidente a simple vista, pero hacer programas eficientes es importante para que permitir la gestión de cantidades mucho mayores de información.